Rollout Cards for Agent Research

Anonymous Author(s)
Affiliation
Address
email

Abstract

Agent research publishes reported numbers — accuracy, cost, wall-clock time, gradient update — without publishing the rollouts those numbers were computed from. Two problems follow. First, numbers are produced by a *reporting convention* applied to each rollout (a grading script, a failure-handling rule, a loss function), and different conventions applied to the same rollouts yield different numbers. Because rollouts are discarded, these divergences cannot be reconciled after the fact. Second, each research community publishes only the aggregates its own conventions compute, so replicating or extending a result along a dimension the original harness ignored requires generating new rollouts entirely.

We audit 50 popular training and evaluation repositories and consolidate 37 documented cross-harness variance pairs across task-success, cost, and timing metrics; none of the audited harnesses surface failed rollouts alongside reported metrics. The same pattern appears in gradient-signal computation during training.

We propose *rollout cards*: an event-sourced publication format in which the rollout is the artefact and any reporting convention can be applied post-hoc. Two experiments evaluate the proposal. The first assesses whether rollout cards let one community recover another community's key metrics from rollouts the original harness would have discarded. The second reconciles the 15.6pp published SWE-agent/Agentless gap on SWE-bench Verified, attributing approximately **[C TBD]**pp to a single failure-classification convention (`no_generation` accounting) rather than method differences.

1 Introduction

Language-model capabilities have grown faster than the research ecosystem evaluating them. The infrastructure that trains and tests these models has fragmented into a zoo of benchmark harnesses, RL trainers, agent frameworks, and observability middleware, each with its own conventions for how a rollout is recorded, how a headline result is computed, and what is silently dropped along the way. A single evaluation now routes a model's behaviour through a stack of four or five ecosystem choices before producing the number that appears in a paper's headline table.

These choices are not cosmetic. Identical LLaMA-65B weights score 63.7 on MMLU under the Berkeley evaluation code and 48.8 under `lm-evaluation-harness`: a 14.9-point gap attributable to prompt-template and answer-extraction conventions alone (**??**). Neither pipeline stores the raw token traces a third party would need to recompute the score under their own convention.

The pattern extends beyond task-success metrics: cost figures diverge by factors of two between tracing frameworks monitoring the same API calls (**??**), wall-clock measurements differ by more than 3× between RL trainers running the same workload (**?**), and training-side research has recently documented gradient-signal differences between RL libraries applying different loss-aggregation conventions to identical rollouts (**?**). Where the underlying rollouts have been preserved, they agree.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

The disagreement enters when each pipeline folds those rollouts into a number; whichever convention the harness happened to ship adjudicates it silently.

We argue that the field has conflated two layers: *recording* (what the rollout captures) and *reporting* (aggregates computed from it). Current practice publishes the aggregate and discards the rollout, which makes every convention disagreement between harnesses a disagreement without a tiebreaker. We propose that the rollout itself be the published artefact.

Contributions. We make three contributions.

- We perform a systematic audit of 50 popular training and evaluation repositories at file-and-line granularity (§**??**). We pin every claim to a specific commit SHA for two-click GitHub verification. **We find that none of the 50 surface failed rollouts alongside their reported metrics. Eleven silently drop failures from their headline numbers, including FastChat (the reference scoring code behind MT-Bench and Chatbot Arena).** We consolidate 37 cases where the same workload yields different published numbers, spanning up to 14.9pp on task success (LLaMA-65B MMLU, Berkeley vs lm-evaluation-harness), 2× on cost (Anthropic cached tokens), and 3.13× on latency (GSM8K-GRPO epoch, TRL vs OpenRLHF).
- **We propose *rollout cards*** (§**??**): a publication-format genre in the Datasheets (**?**) and Model Cards (**?**) lineage, and implement them as Ergon, a reference gym whose event-sourced format τ_E projects into the canonical trajectory formats of five research communities. Each projection ships an explicit manifest of what it drops.
- We evaluate the proposal with two experiments (§**??**). First, we show that we can record rollouts under one community's native convention and re-analyse them under another community's (MiniF2F binary-grading rollouts re-analysed as MCTS tree-search quantities; Research Rubrics scalar-score rollouts as MAS per-agent role differentiation). **Second, we ingest the published SWE-agent and Agentless submissions to SWE-bench Verified, re-grade both under a uniform convention, and find that their 15.6pp published gap decomposes into [C TBD]pp of harness convention (the treatment of no-generation outcomes) and [M TBD]pp of residual method difference.**

The remainder of the paper develops these contributions in order: Section **??** the audit, Section **??** the format and its reference implementation, Section **??** the two experiments.

2 Recording vs Reporting in the Current Ecosystem

Two observations recur across the agent research ecosystem. First, the field disagrees on reported numbers computed from what are, or could be, identical rollouts: pass rates on the same model weights, cost totals on the same API calls, and wall-clock times on the same training runs all differ, sometimes substantially, depending on which *reporting convention* (grading script, aggregation rule, failure-handling rule, timing rule, loss function) a harness applies to the rollout. Second, different research communities want to ask different questions of their rollouts, and the canonical trajectory formats each community publishes reflect which question the community is asking. Taking the two together, *recording* (what the rollout captures) and *reporting* (aggregates computed from it) are distinct layers that should be published separately.

2.1 Convention disagreement across the ecosystem

We consolidate 37 cross-harness variance pairs where the same workload yields different published numbers. The pairs span three metric families (task success, cost/tokens, latency/timing) and hold at every layer of the infrastructure stack; in several cases the disagreement is already documented by the infrastructure projects themselves.

On task success, HuggingFace's Open LLM Leaderboard post-mortem documents a 14.9-point gap on LLaMA-65B MMLU between the Berkeley evaluation code (63.7) and `lm-evaluation-harness` (48.8): identical model weights, identical benchmark split, different prompt-template and answer-extraction conventions (**?**). On cost/tokens, OpenTelemetry PR #3163 formalises a convention disagreement between the OpenAI/Vertex token-accounting model (cached tokens counted inside

input_tokens) and Anthropic's (cached tokens separated into cache_read_input_tokens + cache_creation_input_tokens); the same API request under the two conventions produces published token counts differing by a factor of 2.0×, with the delta affecting downstream cost estimators that read input_tokens as canonical (**?**). On latency/timing, a head-to-head between two RL training frameworks reports 5,189s/epoch (TRL) versus 1,657s/epoch (OpenRLHF) on GSM8K-GRPO with identical hardware, identical model, and identical algorithm; **?**, Table 4 attribute the 3.13× gap to framework-level dispatch conventions rather than algorithmic difference.

Several of these findings are not new. HuggingFace's post-mortem names the MMLU variance explicitly (**?**). NVIDIA NeMo's documentation states that simple-evals GPQA and lm-evaluation-harness GPQA are "distinct, non-comparable metrics" (**?**). OpenTelemetry PR #3163 exists because the cache-token disagreement was reaching production tracing infrastructure (**?**). These acknowledgements live scattered across post-mortems, vendor documentation, and infrastructure-project pull requests; the field has not consolidated them or recognised them as instances of the same phenomenon. Table **??** samples six flagship pairs across the three families; the full 37-pair catalogue is in Appendix **??**.

Table 1: Flagship cross-harness variance pairs across three metric families. Six of 37; the full catalogue with per-pair sources is in Appendix **??**. Δ is the gap between published numbers on an identical workload; the SWE-bench Verified Docker/Modal gap is described by the harness authors as "systematic" rather than a single-number delta (**?**).

Family	**Setup**	**Harness A**	**Harness B**	**Δ**
Task success	LLaMA-65B MMLU 5-shot	Berkeley (63.7)	lm-eval-harness (48.8)	**14.9pp**
Task success	Mistral/Mixtral MMLU	best prompt template	worst prompt template	up to **24.6pp**
Cost/tokens	Anthropic cached API call	OTel input-inclusive	Anthropic separated	**2.0**×
Cost/tokens	Sonnet 3.5 on Aider	Edit benchmark ($0)	Refactor+Polyglot ($14.41)	**$14.41**
Latency/timing	SWE-bench Verified	Docker backend	Modal backend	"systematic"
Latency/timing	GSM8K-GRPO, identical HW	TRL (5,189s/epoch)	OpenRLHF (1,657s/epoch)	**3.13**×

Each pair above is the default behaviour of a widely-used harness applied to widely-used models on widely-used benchmarks, produced without misconfiguration. Where rollouts have been preserved, they agree. The disagreement enters at the heuristic each harness applies at reporting time, and the code paths producing it default to silent behaviour. We audited 50 popular training and evaluation repositories at file-and-line granularity, pinning each to a specific SHA during the April 2026 audit window and scoring its behaviour against five failure scenarios (rollout exception during generation; reward-function exception or None; zero-length or unparseable output; mid-rollout environment failure; process killed mid-rollout). Of the 50 repositories, **none default to surfacing failures alongside metrics**: 11 exhibit catastrophic silent dropping, 31 drop silently, and 9 log failures but exclude them from the denominator with no per-category counter. Four score-3 cases illustrate the pattern. FastChat, the reference implementation behind two of the most-cited public LLM evaluation numbers (MT-Bench and Chatbot Arena), sets rating = -1 on judge parse-failure and then filters df[df["score"] != -1] before computing the per-model mean and Elo, so a model that reliably produces unparseable outputs appears to have no score rather than a bad one.[1]

Three further cases in the same mould. openai/evals retries API errors indefinitely (backoff.on_predicate with no max_tries), so any task that does not time out eventually reports success.[2] PrimeIntellect's verifiers coerces reward-function exceptions to 0.0 and includes them in the group-mean advantage, poisoning the gradient for every other rollout in the group.[3] ToolBench promotes a fraction of give_up_node trajectories to valid_data=True at random, and SFT for ToolLLaMA trains on the promoted failures.[4]

The best-engineered harness in the survey, Inspect AI, defaults to fail_on_error=True and emits per-sample error counters. Under fail_on_error=False (common for long runs) its denominator

[1] fastchat/llm_judge/common.py:L175-L187; show_result.py:L20, :L49; elo_analysis.py:L49-L92 at SHA 587d5cf.

[2] evals/utils/api_utils.py:10-22 at SHA 8eac7a7.

[3] rubric.py:144-158, :208, :217, :325-333 at SHA e27633b.

[4] DFS.py:84-91; preprocess_toolllama_data.py:44-45 at SHA d56fdd8.

becomes non-errored samples with no per-category counter.[5] Even the positive control falls into silent bias under real-world usage pressure.

The pattern extends to training-side computation as well, on identical rollouts: **?** documents +7.3 to +15.7pp swings on AIME 2024 from a single loss-aggregation convention, and verl issue #2165 (**?**) documents a tokenization-channel divergence on Qwen3-4B GRPO reproduced independently by six users. Neither is a rollout-format disparity; both would be auditable if the rollouts were published. We defer the full citation context to Sec. **??**.

2.2 Different communities record different things

Convention disagreement does not happen in a vacuum. Five research communities, five different canonical trajectory formats, each shaped by the questions that community's central analyses need to ask. *Long-horizon LLM-agentic RL* records multi-turn interactions as flat token-concatenated rollouts with per-turn reward attribution, because the community's central analyses are defined over exactly that structure: LOOP's learned behaviours (consulting documentation, minimising confabulation, recovering from setbacks) (**?**) and RAGEN's Echo Trap and reward-variance cliffs (**?**). *Fixed-role multi-agent systems* record per-agent streams with stable identities, because MAST's 14-failure-mode taxonomy over 1,642 annotated traces (**?**) is computed against per-agent attribution and full conversation histories, and its most prevalent category ("specification and system design," 41.8% of failures) turns on per-agent role identification. *Hierarchical and macro-action RL* records step-indexed trajectories with explicit option-termination events, because deliberation cost, option domination, and premature termination (**?**) are functions over those events.

Recursive language models record the agent-call tree with per-node message histories and typed delegation primitives, because ReDel's overcommitment/undercommitment diagnostics (**?**) are functions over the tree's shape. *MCTS-based LLM training* records the search tree with per-node visit counts, Q-values, and abandoned branches, because rStar-Math's process preference model (**?**) trains on preference pairs constructed from exactly those abandoned branches. These five communities are illustrative, not exhaustive: interactive theorem proving, agentic RAG, robot learning, and embodied agents ask their own questions of their rollouts. We focus on these five because Ergon specifies projections for them, with the implemented subset used in §**??** and the remaining projections documented as extension points in Appendix **??**. We expect the same pattern to hold for communities whose projections we have not yet specified.

A shared substrate must therefore preserve what these examples point to: per-agent identity, containment and termination structure, tool-call content with reasoning channels, experimental and environmental context, and first-class failure recording. It must do so while supporting the analyses different communities want to run post-hoc.

3 Ergon: Recording as a Shared Substrate

Section **??** develops the proposal in three parts. Section **??** specifies what a rollout card contains: the streams, the vocabulary, and the extensibility rules. Section **??** introduces *drops manifests*, the methodology for reporting what each reduction of a rollout to an aggregate discarded. Section **??** positions rollout cards in the Datasheets and Model Cards lineage and summarises Ergon as a reference implementation. Full system details (schema, mutation log, trainer adapters, crash durability) are in Appendix **??**.

3.1 What a rollout card contains

A *rollout card* is the portable, publication-ready serialisation of one completed run: a self-describing archive of JSONL streams plus a manifest and an optional blob directory, readable by a downstream consumer without access to the producing runtime. This is the *record* side of §**??**'s recording-vs-reporting decoupling; §**??** describes the *reporting* side. Figure **??** shows one sample from a rollout archive, viewed through the Ergon dashboard (§**??**); its panels preview the format's components before we name them. The bundle is medium-independent — distributable as a directory, a zip, or a

[5] Inspect AI at SHA `36231d6`.

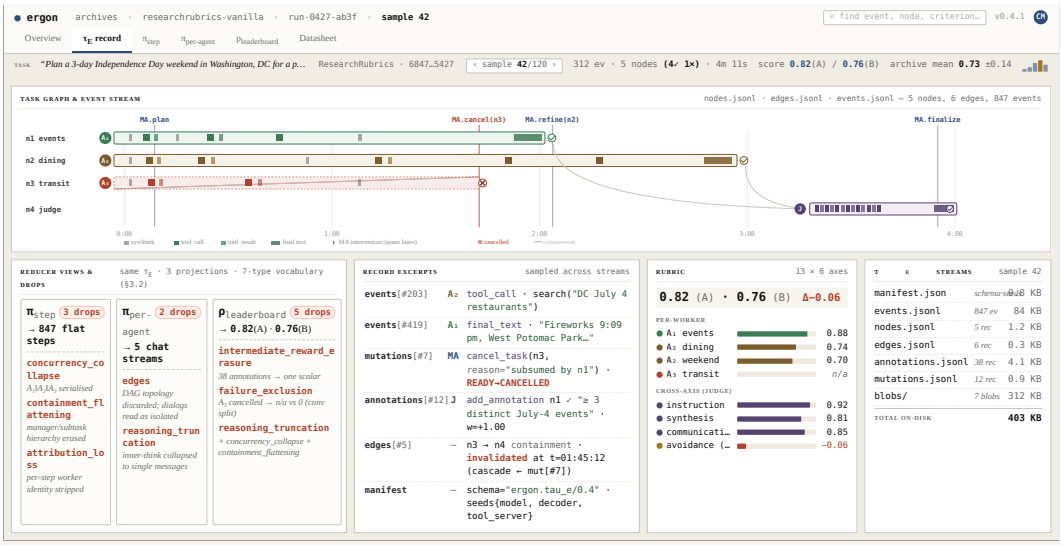

Figure 1: One sample from a rollout card, viewed through the Ergon dashboard. A τ_E record is a bundle of seven JSONL/JSON artefacts on disk. *Top:* the sample's task-execution DAG with per-worker swim lanes and manager-agent interventions as annotations. *Bottom:* three reducer views (π_{step}, $\pi_{\text{per-agent}}$, $\rho_{\text{leaderboard}}$) with their drops manifests (§**??**), stream excerpts, the rubric decomposition producing the reported scalar, and on-disk stream sizes. The dashboard is one consumer of the format; any τ_E-conformant archive loads (§**??**).

HuggingFace dataset, and implementable over any storage backend that preserves the row semantics of Appendix **??**.

Each stream addresses one of the five recording needs of §**??**. The event stream records the reasoning-and-action trace on a monotonic per-task-execution sequence axis as a typed vocabulary of events (full payload union in Appendix **??**). The node stream is the containment tree of task-execution nodes, each carrying a parent pointer, a worker identity stable across the node's lifetime, and a status drawn from an ontology declared per-run in the annotation namespace. The edge stream holds the dependency DAG separately from the parent-child tree, so fire-and-forget delegation and sibling-diamond dependencies are both representable. The annotation stream is a namespace-keyed versioned store that reserves one namespace for task payloads and leaves all others to experiment code, projections, or external instrumentation. The mutation stream records structural change on the same sequence axis as events, drawn from a closed nine-value vocabulary (Appendix **??**); failures appear here as status transitions rather than exceptions routed around the recording path, the mechanism underneath §**??**'s 0-of-50 failure-surfacing finding. A run manifest carries metadata, a format version, and content hashes; a blob store holds content-addressed overflow for payloads above an inline size cap (default 64 KB). The five streams cover the community needs of §**??** directly: per-agent identity and failure attribution (**??**), structured reasoning-action traces (**??**), containment and termination semantics (**??**), and concurrent dispatch.

The format is lossless: every row in the streams corresponds directly to a row persisted during the run, and downstream analyses read whichever streams and namespaces they recognise. Extensibility is via namespaces — a team studying cross-agent tool negotiation can claim a `tool-negotiation` namespace and record per-edge metadata that other consumers simply ignore, and the card remains a valid card. Ergon ships a reference exporter (`ergon export-rollout-card <run_id>`), a Pydantic validator (`ergon validate-rollout-card <path>`), and publishes one example card per demonstration of §**??** as a HuggingFace dataset with the camera-ready codebase.

3.2 Reporting what was dropped

Publishing the rollout is half of the reconciliation story; the other half is publishing what was discarded in going from rollout to reported number. A *drops manifest* is the typed-vocabulary record accompanying any reduction of a rollout to an aggregate: a list of entries naming the erasure type, the

rollout element it points at, and the reported quantity the removal affects. We draw erasure types from a small vocabulary covering the erasures Sec. **??** documented in practice: failure exclusion, cache-accounting convention, intermediate-reward erasure, concurrency collapse, containment flattening, reasoning truncation, and attribution loss. Figure **??**'s reducer-views panel displays the drops manifests for three projections over the sample shown: π_{step} records three erasure classes (concurrency collapse, containment flattening, intermediate-reward erasure), $\pi_{\text{per-agent}}$ records two, and $\rho_{\text{leaderboard}}$ records five. This addresses Sec. **??**'s diagnoses directly: 0 of 50 audited repositories surface failures alongside metrics because they do not record that failures were excluded; the 37 cross-harness variance pairs disagree untraceably because neither harness records which erasures its own convention applied. Making reductions legible requires naming the erasures. Drops manifests are additive to any reducer: a team re-grading SWE-bench trajectories under their own convention can ship a drops manifest without adopting Ergon, and a reviewer auditing two harnesses' disagreement on the same rollout can compare their drops manifests directly. Ergon's projection operators (Appendix **??**) are one instantiation, shipping a drops manifest per target format; the vocabulary itself is the proposal.

3.3 Ergon: a reference implementation

We implement the format as Ergon, an async gym with a persistent recording substrate. Ergon hosts existing agent frameworks (LangGraph, CrewAI, AutoGen, Claude Code) via lightweight adapters and feeds RL trainers (TRL, VERL, OpenRLHF) through format projections that preserve the full trajectory underneath. Ergon writes streams synchronously under Postgres transactions; the `ergon export-rollout-card <run_id>` command emits a rollout card from any completed run, and the Figure **??** dashboard is a reference reader over any τ_E-conformant archive. Ergon is one implementation of the format: any framework matching the specification (Appendix **??**) can publish rollout cards. Adoption is additive: a paper can ship cards without changing its headline table, and later work can re-analyse them under whatever convention is useful. The paper still reports the scalar; it also publishes the rollout; the convention that produced the scalar is separable, inspectable, and replaceable.

4 Experiments

4.1 Research questions

We evaluate the proposal with two experiments, each addressing one of the two problems diagnosed in §**??**: RQ1 targets the cross-community recording mismatch of §**??**, RQ2 targets the convention-disagreement reconciliation of §**??**. Both use rollout cards as a shared artefact against which we apply conventions post-hoc; neither requires re-running agents. The experimental scope is a proof of concept. Two pairings in RQ1 and one reconciliation in RQ2 are enough to establish that the format is structurally richer than any single community's canonical format. We do not claim that this two-pairing scope exhausts the community pairs for which the format is adequate.

RQ1: Cross-community reanalysis. Can a rollout card recorded under one research community's native scorer be consumed by a different community's scorer to recover a quantity the original publication did not report? Positive evidence would show that the format is rich enough to serve communities whose questions differ from the recording community's, directly from the rollout rather than by re-generating trajectories under a new harness.

RQ2: Cross-harness reconciliation. Can published trajectories from two harnesses reporting different scores on the same benchmark be ingested into a common rollout card format, re-graded under a uniform convention, and the reported score gap decomposed into harness-attributable and method-attributable components? Positive evidence would show that rollout cards provide a substrate for making otherwise untraceable reported-number disagreements auditable.

4.2 Setup

The experiments use three task families. For the first two (MiniF2F and Research Rubrics) we run a flexible-decomposition agent natively in Ergon to produce rollouts; these exercise RQ1. For the third (SWE-bench Verified) we ingest rollouts from two published submissions (SWE-agent and Agentless,

both on GPT-4o); this exercises RQ2. The two experiments draw on six behavioural quantities, each a canonical analysis of one of the five communities of Sec. **??**; Appendix **??** catalogues the full set and we compute three in §**??**. Ergon commits every mutation synchronously (Appendix **??**), so a mid-rollout crash leaves the run graph in a well-defined partial state.

Cross-community reanalysis (MiniF2F, Research Rubrics). A single *flexible-decomposition agent* runs on both task families with a shared backbone (**[model_target TBD]**) and a turn cap of **[N TBD]**. Its system prompt describes four subtask-decomposition tools (spawn, cancel, wait-on, and report-result) mechanically, with no task-specific strategic guidance, so observed decomposition structure comes from the agent rather than the prompt. Ergon records trajectories directly as rollout cards (Sec. **??**).

MiniF2F ($\sim$30 problems, Lean 4 theorem proving (**?**)) uses `lean_repl` / `lean_check` and binary grading. **Research Rubrics** ($\sim$30 questions, long-horizon open-ended research) uses `web_search` / `read_document` with rubric-weighted GPT-4o-mini evaluation.

For RQ1 we pair each task family with a community whose native scorer was *not* the one the task's designers had in mind. MiniF2F, which we record for binary proof-success, pairs with the MCTS-based training community (**??**), whose quantities include abandonment ratio by tree depth and sub-goal semantics at abandoned nodes. Research Rubrics, which we record for scalar rubric scores, pairs with the fixed-role MAS community (**?**), whose quantities include per-agent role specialisation; we capture it via sibling-embedding distance across workers the agent spawns from the same parent. In each pair, the target quantity is one the source scorer discards but the recorded rollout carries.

Cross-harness reconciliation (SWE-bench Verified). The paper's motivating gap is SWE-agent's 23.2% vs. Agentless's 38.8% on GPT-4o-2024-05-13 (15.6pp). Both submissions publish complete trajectories through `swe-bench-submissions`; we ingest each into a rollout card and re-grade under a uniform convention. The ingestion is the main engineering step: SWE-agent ships per-instance JSON with a flat conversation history and a denormalised `trajectory` array, Agentless ships per-instance Python-logger plaintext with `ChatCompletion(...)` reprs across a seven-stage pipeline. Each ingestion produces a drops manifest (Appendix **??**) naming what the source format could not carry, exercising the methodology of Sec. **??**.

One convention choice matters enough to foreground: the treatment of `no_generation` outcomes, where the harness records no agent submission. SWE-agent produces 50 such outcomes (10% of Verified); Agentless produces 4 (0.8%). We report under two conventions. *Convention A* excludes no-generation cases from the denominator (isolating attempts from non-attempts); *Convention B* includes them as zeros (what the published headlines implicitly use). The choice surfaces a harness-level decision that neither published paper's headline makes visible.

4.3 Results

Results come in three parts: RQ1a recovers MCTS-community quantities from MiniF2F rollouts, RQ1b recovers MAS-community quantities from Research Rubrics rollouts, and RQ2 decomposes the SWE-agent/Agentless gap on SWE-bench Verified.

RQ1a: MiniF2F $\rightarrow$ MCTS-community quantities. We record **[N TBD]** MiniF2F rollouts under binary grading and recover abandonment ratios as a function of containment depth: at depth 1 decompositions, **[A_1 TBD]**% are abandoned before completion; at depth 2, **[A_2 TBD]**%; at depth 3, **[A_3 TBD]**%. Clustering the assistant-text content at abandoned nodes yields **[K TBD]** distinct sub-goal categories (e.g. **[category descriptions TBD]**), matching the negative-preference training signal rStar-Math's PPM uses (**?**). We cannot derive either quantity from the binary pass/fail the MiniF2F headline reports; we compute both post-hoc from the rollout card's node and event streams without re-running any proofs.

RQ1b: Research Rubrics $\rightarrow$ MAS-community quantities. We record **[N TBD]** Research Rubrics rollouts under rubric-score grading and compute per-agent sibling embedding distance: **[D TBD]**% of sibling pairs have cosine distance above **[threshold TBD]**, indicating role differentiation in the sense of MAST's duplicate-agent failure mode (**?**). The rubric-scored headline reports a scalar per question; we can compute the role-specialisation analysis from the same rollouts, and **[Q TBD]** of **[N TBD]**

exhibit the duplicate-agent pattern invisible in the rubric number. Neither task family was designed for the community whose quantities we recover, so we claim only that the relevant quantities are *computable* from the rollouts (which current practice of publishing community-native aggregates denies), not numerical parity with a native harness.

RQ2: Cross-harness reconciliation on SWE-bench. Approximately **[convention TBD]**pp of the 15.6pp published SWE-agent/Agentless gap on SWE-bench Verified resolves to a single harness convention — the treatment of `no_generation` outcomes, where SWE-agent produces 50 such instances (10% of Verified) and Agentless produces 4; the remaining **[M TBD]**pp is method-attributable. Neither publication's reported number makes this classification choice visible. Table **??** decomposes the gap: under Convention B (the published convention, treating no-generation as zero) the re-graded gap is **[gap_B TBD]**pp; under Convention A (excluding no-generation from the denominator) the remaining gap is **[gap_A TBD]**pp, attributable to how each harness elicits patches. We take the decomposition itself to be the primary deliverable: a gap that published as a pure method difference turns out to have a substantial harness-convention component, invisible without the rollout-card substrate.

Table 2: Decomposition of the 15.6pp published gap between SWE-agent and Agentless (GPT-4o-2024-05-13, SWE-bench Verified). Under Convention B, no-generation outcomes count as zeros (the published default); under Convention A, they are excluded from the denominator. Ingestion drops manifests are in Appendix **??**.

	SWE-agent	**Agentless**
Published score	23.2% (116/500)	38.8% (194/500)
Re-graded, Convention B	**[X TBD]**%	**[Y TBD]**%
Re-graded, Convention A	**[X' TBD]**%	**[Y' TBD]**%
Published gap	15.6pp	
Convention-attributable (B → A delta)	**[C TBD]**pp	
Method-attributable (under A)	**[M TBD]**pp	

A second observation cuts the other way. Agentless at SHA `5ce5888` defaults missing regression-test results to `[0]*10000` rather than surfacing them as failures (Appendix **??**, Patterns P1 and P4), so instances where its own pipeline errors out are charged against the method. The silent-drop bias therefore runs in the wrong direction to absorb the convention gap: if anything, it *amplifies* the 15.6pp. The audit pins `agentless/test/run_regression_tests.py` at the named SHA, and a reader can verify the `[0]*10000` default in two clicks. That SWE-agent JSON and Agentless plaintext-log formats both ingest into the same target with documented drops manifests (Appendix **??**) shows the rollout-card format accommodates heterogeneous sources, not only data recorded natively.

5 Related Work

We position rollout cards against four adjacent threads: the documentation-standards genre they extend, convention-variance documentation in evaluation and training, the environment-and-framework landscape Ergon slots into, and schema-alignment efforts at training-input time.

Genre precedent. Rollout cards sit alongside Datasheets for Datasets (**?**) and Model Cards for Model Reporting (**?**): structured artefacts published alongside a research object to make downstream use legible. Sec. **??** adapts the pattern to agent trajectories.

Convention variance in adjacent spaces. **?** find 24.6pp variance on LLaMA-7B MMLU across three popular harnesses driven by prompt-template choice. On the training side, **?** document +7.3 to +15.7pp on AIME 2024 from a single GRPO loss-aggregation convention, and verl issue #2165 (**?**) documents tokenization-channel divergence on Qwen3-4B GRPO with six independent reproductions. Those works identify and fix specific conventions; the present work proposes infrastructure for making future disagreements of this kind auditable before they calcify.

Environment and framework landscape. Gym (**?**) and Gymnasium (**?**) are the canonical precedent for a shared evaluation interface in RL; Ergon makes the same move at a higher level

of abstraction, standardising recording, rollout execution, and trainer integration for decomposed long-horizon work. PettingZoo (**?**) and JaxMARL (**?**) extend Gym to fixed-population multi-agent with joint actions; dynamic population, asynchronous dispatch, mid-rollout cancellation, and non-tree dependency sit outside that paradigm. Agent frameworks (LangGraph (**?**), CrewAI (**?**), AutoGen (**?**), Claude Code (**?**)) and inference-time recursive toolkits (ReDel (**?**), RLM (**?**), AgentOrchestra (**?**)) run under Ergon's substrate with automatic recording.

Schema alignment efforts. Recent surveys (**???**) flag fragmentation across agent trajectory formats, and unification efforts address it at training time: AgentOhana (**?**) standardises trajectories into a unified training loader, VerlTool (**?**) aligns tool-integration logic across agentic RL codebases, and Agent Lightning (**?**) decouples LLM generation from agent logic. Those proposals unify *training inputs*; we propose a *publication format*, with drops manifests (Sec. **??**) supplying a per-format account of what each canonical shape cannot represent.

6 Discussion

We discuss four aspects of the proposal in turn: publication cost and the adoption path, the distinction between detection and resolution, scope limitations of the present work, and future directions.

Publication cost and adoption. Publishing rollout cards is orders of magnitude cheaper than hosting models: a rollout card for a completed agent run is at most a few megabytes of text and event records, where a model checkpoint is tens to hundreds of gigabytes. The precedent for domain-wide rollout publishing exists elsewhere: CERN Open Data (**?**) releases detector-event records from LHC experiments; genomic consortia (1000 Genomes, UK Biobank) publish per-individual read-level data alongside aggregate findings; astronomy surveys (SDSS, Gaia) publish observation-level exposures with derived catalogues. In each case the shared artefact is the observation, not only the analysis. Agent research can make the same move. Adoption is opt-in per researcher and requires no coordinated ecosystem pivot: a paper that ships a rollout card alongside its headline number loses nothing and gains auditability; a reader who wants to reanalyse can do so against the rollout without asking for the original harness.

Detection over resolution. Rollout cards let a reader *detect* convention disagreement. They do not by themselves force a *resolution*. A second researcher applying a different convention to the same rollouts will generally produce a different number than the original paper, and we consider this a feature: the disagreement becomes legible, because both conventions' drops manifests are publishable and comparable, and reconcilable, because a uniform convention can be applied to both. The Datasheets and Model Cards analogy holds here too: documentation standards make practice legible; they do not force methodological convergence. We expect a field that adopts rollout cards to have more transparent disagreements than it has today.

Limitations. Coverage is representative: we draw five communities from a long tail, audit 50 repositories from a much larger population, and select 37 variance pairs from the subset where documentation made comparison possible. We cite the training-side convention-variance evidence (Dr. GRPO, verl #2165) rather than reproduce it here. The cross-community reanalysis is single-analyst: we pair each task family with a single target community rather than a multi-community panel. The cross-harness reconciliation demonstrates the format on two published submissions on one benchmark; we leave generalisation across benchmarks and model variants to future work. We claim the general pattern (conventions applied without rollouts make disagreement untraceable) is robust across these scope bounds, but not that every specific number generalises.

Future work. Two directions. First, convention-provenance tooling: a rollout card plus its drops manifest is a consumable artefact for automated variance analysis; a tool that ingests both and flags convention divergences would make the SWE-bench finding of Sec. **??** routine rather than ad hoc. Second, training-side rollout cards: Sec. **??**'s training-side evidence implies an analogue at the gradient-computation layer where the loss and advantage rules play the role of the reporting convention; we have not worked out what that looks like.

7 Conclusion

Agent research publishes reported numbers without the rollouts those numbers were computed from, and the resulting coupling between recording and reporting produces both cross-community fragmentation and cross-harness variance that cannot be reconciled after the fact. We proposed *rollout cards*: an event-sourced publication format in which the rollout itself is the research artefact, and a later researcher can apply any reporting convention (grading script, aggregation rule, training loss) post-hoc. We introduced drops manifests as a paper-level methodological concept accompanying any reduction of a rollout to an aggregate, and presented Ergon as a reference implementation at the intersection of the agent and RL stacks. Two experiments demonstrated the proposal: cross-community reanalysis on MiniF2F and Research Rubrics rollouts showed that recorded trajectories carry quantities the recording community's scorer discarded, and cross-harness reconciliation on SWE-bench Verified showed that a substantial fraction of a widely-published score gap resolves to a harness convention invisible in the reported numbers. If the field adopts the format, reported-number disagreements become legible and auditable. That is the tractable precondition we argue for: an ecosystem in which the field can compare cross-community and cross-harness results on their merits rather than on the conventions of their producers.

A 50-repo Failure-Handling Survey

This appendix reports the code-level evidence underlying the failure-surfacing claim in Sec. **??**. The final sample contains 50 in-scope repositories selected by category-stratified purposive sampling across the code paths by which LLM-agent and LLM-training papers publish benchmark numbers, training metrics, or leaderboard ranks. Each repository was shallow-cloned at a pinned commit SHA during the April 2026 audit window and scored against a seven-pattern failure-handling rubric. Every source-level claim below is backed by a commit-pinned GitHub permalink in the supplementary survey document `SURVEY_v4.md`; pinned SHAs are listed in §**??**.

A.1 Repository selection

Sampling frame and operational scope filter. The survey uses *category-stratified purposive sampling* over the code pipeline through which an LLM-agent paper publishes a benchmark number. A repository is in-scope if and only if, at its pinned SHA, it contains code that (a) is actively maintained (last commit within the past 12 months), (b) produces a benchmark number, training-pipeline metric, or leaderboard rank that a published paper would cite, and (c) implements the result aggregation itself rather than delegating all aggregation to a separately-versioned downstream harness. Star-count was not used as a threshold; verified in-scope repositories range from ~ 150 stars (SciCode 164, MLAgentBench 320) to ~ 40k+ (FastChat, aider), demonstrating that the in-scope/canonical-harness filter dominates any star-based floor. Operational scope is enforced by two exclusions, documented as examples rather than concealed: Online-Mind2Web was on the candidate list but excluded after repeated clone errors prevented SHA pinning; OpenHands was spot-checked (SHA `3b17f27`) and documented OUT-OF-SCOPE because its evaluation harness lives in a separately-versioned repository (`github.com/OpenHands/benchmarks`) and the main repo contains no result-aggregation, scoring, or denominator-handling code. This is the operational scope filter working as intended: OpenHands is the foundation layer (analogous to how PyTorch is the foundation for training frameworks) and is correctly excluded by criterion (c).

Sample composition. The 50 repositories are grouped into five strata chosen to cover the major places where agent-research numbers are produced: *(i)* SWE-bench-family harnesses and agent scaffolds; *(ii)* RL and agentic-RL training frameworks; *(iii)* general evaluation harnesses, LLM-as-judge systems, and leaderboard infrastructure; *(iv)* web, GUI, mobile, scientific-agent, and ML-engineering benchmarks; and *(v)* function-calling, multi-agent, and general agent-scaffold repositories. Selection was driven by operational role rather than popularity: a repository was included when it contained aggregation, scoring, denominator handling, failure handling, reward computation, or training-metric code that could affect a number cited in a paper.

Repository strata. *(i)* SWE-bench-family harnesses and agent scaffolds (SWE-bench, SWE-agent, mini-swe-agent, live-swe-agent, SWE-bench_Pro-os, SWE-smith, Agentless, aider); *(ii)* RL train-

ing frameworks (TRL, verl, OpenRLHF, rllm, slime, ART, Agent-R1, RAGEN, MARTI, MATPO, RL-Factory, Trinity-RFT, ms-swift, verifiers, open-r1); *(iii)* general eval harnesses and LLM-as-judge leaderboards (openai/evals, simple-evals, SciCode, lm-evaluation-harness, lighteval, inspect_ai, FastChat/MT-Bench + Chatbot Arena, HELM, ragas, deepeval, MTEB, promptfoo, BIG-bench); *(iv)* web/GUI, mobile, and ML-engineering / scientific-agent benchmarks (WebArena, VisualWebArena, OSWorld, android_world, Mind2Web, MLE-bench, MLAgentBench, ScienceAgentBench); and *(v)* function-calling and general multi-agent / scaffold repositories (BFCL/gorilla, ToolBench, AgentBench, AgentBoard, tau-bench, Self-rewarding-reasoning-LLM). The final sample is 50 in-scope repositories, with two exclusions (Online-Mind2Web, OpenHands) documented as scope-filter exemplars rather than concealed.

A.2 Audit methodology

Each repository was shallow-cloned once into `ergon_survey/clones/{repo}` and pinned at `git rev-parse HEAD`. The pinned SHA list in §**??** is the verification baseline for every permalink. Every citation in this appendix resolves to `github.com/{owner}/{repo}/blob/{SHA}/{path}#L{start}-L{end}` against that SHA. Each claim identifies the code path that controls failure handling, denominator construction, reward coercion, or result aggregation for a number the repository can publish. Two representative per-repo reports are included verbatim in the supplementary materials to illustrate the template.

A.3 Severity rubric: seven patterns of silent drop

Every repository was scored 0–3 against a rubric defined by seven recurring code patterns: *(P1)* `None`-to-`0.0` reward coercion (reward-function exception silently yields a numeric zero that enters aggregation alongside genuine zero-reward rollouts); *(P2)* variance-based rollout filters that discard entire rollout groups with zero std (collapse with the remaining group in published aggregates); *(P3)* save-on-clean-exit patterns bypassed by `KeyboardInterrupt` or early-exit cost-killers, removing instances from the submission file rather than the denominator; *(P4)* survivor denominators — reporting `sum(scores) / len(scores)` over instances that reached the aggregation step, with the pre-aggregation drop count not preserved; *(P5)* multi-turn N-destroys-1-to-N-1 patterns, where a mid-rollout tool-call exception discards the entire trajectory including its already-completed turns; *(P6)* LLM-judge regex-miss-to-fixed-score, where a judge returning an unparseable response is coerced to a specific numeric score indistinguishable from a legitimate one (e.g. `0.0`, `min(choice_scores)`, `NOT_ATTEMPTED`); and *(P7)* outcome-as-ground-truth SFT filtering, where a downstream supervised dataset is built from only those trajectories that scored correctly on the biased harness, propagating the bias across a model-generation boundary. A repository scores 3 if a single pattern is catastrophic and observable at the headline output (the failure changes the number a paper would cite); 2 if one or more patterns are present but the reader cannot tell from the published CSV; 1 if failures are logged but excluded from the denominator with no per-category counter; 0 if all failures are surfaced alongside metrics. No repository in the survey defaults to score 0.

A.4 Score distribution

Inspect AI (score 1) is the positive control referenced in Sec. **??**: its default `fail_on_error=True` surfaces per-sample errors, but the commonly-used `fail_on_error=False` opt-in degrades silently with no per-category counter (`inspect_ai/_eval/task/error.py:L26` and `inspect_ai/scorer/_metrics/accuracy.py:L33` at SHA `36231d6`).

A.5 Per-repo entries

We summarise each repository's score with a one-line evidence pointer; the eleven score-3 repositories receive a short 2–3-sentence writeup naming the specific code site(s) and user-visible consequence. All paths are relative to the repo root at the pinned SHA (§**??**). Full permalinks are in `SURVEY_v4.md`.

Score-3 repositories (11). SWE-bench. The grading loop aggregates over `instances_to_run`, filtered before aggregation (`swebench/harness/run_evaluation.py:L465-L470`), while empty patches are documented as filtered pre-submission (`docs/faq.md:L39`). A container-level infras-

Table 3: 50-repo severity scores. No repository defaults to surfacing failures alongside metrics. OpenHands is documented OUT-OF-SCOPE (no in-repo aggregation harness; benchmarks live in a separately-versioned repository) and not counted in the 50. Simple-evals appears in both score-3 (SimpleQA subset) and score-1 (non-SimpleQA subset); the $11+31+9=51$ listings correspond to 50 unique repositories.

Score	Count	Repositories
3	11	SWE-bench, openai/evals, simple-evals (SimpleQA), VisualWebArena, ToolBench, Self-rewarding-reasoning-LLM, PrimeIntellect verifiers, Trinity-RFT, ms-swift (GRPO), BFCL (gorilla), FastChat (MT-Bench + Chatbot Arena)
2	31	Agent-R1, MARTI, SWE-bench_Pro-os, RAGEN, OpenRLHF, TRL, ART, slime, tau-bench, rllm, live-swe-agent, SWE-agent, mini-swe-agent, MATPO, verl, AgentBoard, AgentBench, RL-Factory, SWE-smith, lm-evaluation-harness, WebArena, MLAgentBench, MLE-bench, lighteval, Agentless, aider, Mind2Web, open-r1, ragas, MTEB, promptfoo
1	9	SciCode, inspect_ai, ScienceAgentBench, android_world, OSWorld (harness), simple-evals (non-SimpleQA subset), HELM, BIG-bench, deepeval
0	0	—

tructure failure stays in the denominator as a fail, but an empty-patch instance is removed from the denominator altogether — and no output field separates the two cases in the published submission.

openai/evals. `backoff.on_predicate` at `evals/utils/api_utils.py:L10-L22` retries transient predicates indefinitely with no `max_tries`, so long runs silently absorb retry-exhaust events rather than surfacing them. The MMMU eval treats errors as wrong answers (`elsuite/mmmu/eval.py:L158`) and the modelgraded classifier coerces `INVALID_STR` to `min(choice_scores)` (`elsuite/modelgraded/classify_utils.py:L99-L101`): three distinct P6 patterns in one library.

simple-evals (SimpleQA subset). The regex fallback at `simpleqa_eval.py:L126` defaults unparseable responses to `NOT_ATTEMPTED`, which the headline metric (`accuracy_given_attempted = correct / (correct + incorrect)`) then structurally excludes (`simpleqa_eval.py:L169-L176`). A model that hedges on every ambiguous question looks arbitrarily accurate under this metric — NOT_ATTEMPTED tasks do not appear in the denominator.

VisualWebArena. An `assert "correct" in response` at `browser_env/helper_functions.py:L606` raises on unparseable judge output, causing the outer try/except at `run.py:L453-L461` to drop the entire task. Image-fetch errors at `image_utils.py:L44` propagate through the same path. Dropped tasks are absent from the output, not counted as failed.

ToolBench (9/9 VERIFIED). The DFS search code at `DFS.py:L84-L91` randomly promotes a fraction of `give_up_node` trajectories to `valid_data=True`, and SFT preprocessing at `preprocess/preprocess_toolllama_data.py:L44-L45` filters on `valid_data` alone — so ToolLLaMA trains on the promoted failures. This is the paper's paradigm example of pattern P7 (outcome-as-ground-truth SFT).

Self-rewarding-reasoning-LLM (4/4 VERIFIED). `infer_math/reward_labeling.py:L1612-L1622` grants correctness against the ground-truth answer on the *initial* chain-of-thought; self-correction attempts must match that `first_reward` to be counted (`process_prompt_turn3.py:L24-L62`), with a hard cap $N = 3$. There is no `try/except` anywhere in the generation module (`gen_hf.py`): any infra failure silently removes a rollout.

PrimeIntellect verifiers (5/5 VERIFIED). Reward-function exceptions at `rubric.py:L144-L158` yield `0.0`, and group reward-function exceptions at `rubric.py:L208, L217` yield `[0.0]*N`. The er-

rored zeros are then included in the group-mean advantage at `rubric.py:L325-L333` (`avg_reward = sum(aggregated_rewards) / num_states; t["advantage"] = state["advantage"]`), so an infrastructure error in one rollout biases the policy gradient for the other rollouts in its group.

Trinity-RFT. `asyncio.TimeoutError` at `trinity/explorer/scheduler.py:L214-L216` is packaged into `Status(ok=False, metrics=list(), message=...)`, but the consumer path at `scheduler.py:L532-L602` (`get_results`) never inspects `status.ok` — errored payloads flow into training with `ok=False` undetected. The over-rollout cancellation grace period (`scheduler.py:L559-L572`, 30 s default from `config.py:L147`) means timeouts can also truncate sub-tasks mid-run (`scheduler.py:L490-L506`) without surfacing the partial-completion to the rollout log.

ms-swift (GRPO) (8/8 VERIFIED). None-rewards are lifted to NaN at `swift/rlhf_trainers/grpo_trainer.py:L359, L367, L379`, then aggregated with `.nansum(dim=1)` (`grpo_trainer.py:L473`); DAPO's `max_resample_times` revert (`grpo_trainer.py:L684-L733`) and `overlong_filter` mask-zeroing (`grpo_trainer.py:L1123-L1129`) both silently remove rollouts from the gradient. The LLM-judge path (`rewards/rm_plugin.py:L216-L226`) coerces regex-parse failures to `0.0` (pattern P6).

BFCL / gorilla. The weighted overall accuracy formula in `berkeley-function-call-leaderboard/bfcl_eval/eval_checker/eval_runner_helper.py:L509-L519` applies $[10, 10, 10, 30, 40]$ weights over category runners that each silently decrement `correct_count` on parse failure while leaving the denominator `len(model_result)` unchanged (per-category dispatch at `eval_runner.py:L668`). `eval(func_call)` exceptions become literal `"Error during execution: ..."` strings (`multi_turn_eval/multi_turn_utils.py:L97-L98`), scored as a failed call without category attribution.

FastChat (MT-Bench + Chatbot Arena). FastChat is the reference implementation for two of the most-cited public LLM evaluation numbers: MT-Bench (LLM-as-judge single-turn and multi-turn scoring) and Chatbot Arena (pairwise Elo ranking). A GPT-4 or Claude judge whose response fails to match the `[[score]]` or `[score]` regex is coerced to `rating = -1` (P6) at `fastchat/llm_judge/common.py:L175-L187`; MT-Bench aggregation then filters `df[df["score"] != -1]` before computing the per-model mean (P2+P4) at `show_result.py:L20`. The pairwise path is structurally identical: parse-failure becomes `winner = "error"` (`common.py:L282-L304`), filtered out before Elo computation (`show_result.py:L49`; `elo_analysis.py:L49-L92`). A rank swap driven by differential parse-failure rates across judged models is not recoverable from the headline MT-Bench or Arena Elo number — the attrition count is not emitted. At SHA `587d5cf`.

Score-2 repositories (31, one-liner each).

SWE-agent: cost/kill `preds.unlink()` on `early_exit/None` removes instances from `preds.json` (`sweagent/run/run_batch.py:L397-L401`).

mini-swe-agent: `finally: save()` at `src/minisweagent/run/benchmarks/swebench.py:L171` is bypassed by `KeyboardInterrupt` (closed issue #329); streaming save at `default.py:L94`.

live-swe-agent: configs never set `output_path` (`README.md:L71`).

SWE-bench_Pro-os: `None` return at `evaluation/swe_bench_pro_eval.py:L346-L349` is collapsed to `False` by callers at `:L490-L505, L571`.

AgentBench: accuracy denominator is `result.error is None` only (`src/assigner.py:L368-L372`).

AgentBoard: timeouts write to `error.txt` and never `scores.append` (`agentboard/tasks/webbrowse.py:L275-L279`).

tau-bench: errors yield stubs with no retry (`tau_bench/run.py:L89-L96`).

SWE-smith: broad `except` at `scripts/collect_trajs.py:L77-L79`; `random.sample` cap at `scripts/combine_trajs.py:L86-L91`.

TRL: `None`→NaN→`nansum` at `trl/trainer/grpo_trainer.py:L1228-L1230, L2132`.

verl: PRIME reward timeouts→`0.0` at `verl/workers/reward_manager/prime.py:L37-L42, L83`; missing `return_exceptions=True` at `verl/experimental/agent_loop/agent_loop.py:L603` (closed issue #5956).

OpenRLHF: remote RM exception→`reward=None` (`openrlhf/utils/agent.py:L298-L299`; open issue #1139).

rllm (Berkeley Sky): no-valid-trajectory episodes dropped at `rllm/engine/agent_workflow_engine.py:L266`; `dropped_episodes` never written into `DataProto.meta_info` at `:L249` (open issue #382).

slime: oversampling discard + zero-std filter at `slime/rollout/sglang_rollout.py:L449`.

ART: `drop_zero_advantage_trajectories=True` default at `src/art/preprocessing/tokenize.py:L158`.

Agent-R1: length-truncated trajectories published as complete at `agent_r1/agent_flow/agent_env_loop.py:L120-L129`.

RAGEN: variance filter skips whole training step at `ragen/trainer/agent_trainer.py:L1054-L1056`.

MARTI: dynamic filter drops saturated groups at `marti/trainer/ppo_trainer.py:L481-L510`.

MATPO: MCP + judge failures→reward 0.0 at `verl/tools/mcp_tool.py:L137, L150` and `llm_judge.py:L265, L312, L356`.

RL-Factory: malformed tool JSON→`continue` at `verl/workers/rollout/sglang_rollout/sglang_rollout.py:L914-L916`; PRIME timeouts→0.0 at `prime.py:L141-L146`.

lm-evaluation-harness: left-truncate with `eval_logger.warning` only (`lm_eval/models/huggingface.py:L1360-L1368`; closed issues #3419, #3352, #3161, #1323).

WebArena: outer `except Exception: log-and-continue` at `run.py:L217-L364`; headline `sum(scores) / len(scores)` over survivors at `run.py:L365`; `env.step` swallow with `terminated=False` on infra failure at `browser_env/envs.py:L239-L248`.

MLAgentBench: subprocess crash→`"EnvError"` string at `low_level_actions.py:L181-L220` + `environment.py:L328-L334`; hard-coded magic baselines at `plot.py:L249-L274`; fresh random GT per call at `fathomnet/eval.py:L11`.

MLE-bench: aggregation `pad_missing=False` drops incomplete seeds at `experiments/aggregate_grading_reports.py:L69-L135`; `MedalInfo` logic at `mlebench/grade_helpers.py:L123-L133`.

lighteval: LiteLLM returns empty `LitellmModelResponse()` on Azure content-filter and retry-exhaust (`litellm_model.py:L243, L254`); empty response scored 0 at `metrics_sample.py:L151-L152`; left-truncate prompts without `truncated_tokens_count` at `vllm_model.py:L374-L397`.

Agentless: missing regression-test results default to `[0]*10000` at `agentless/test/run_regression_tests.py` (P1+P4), collapsing absent results into a vector of 10k zeros; `_post_process_multifile_repair` returns an empty tuple on exception, and SWE-bench filters empty patches pre-submission (`docs/faq.md:L39`), removing the instance from the denominator. Directly relevant to §**??**'s RQ2 reconciliation of the SWE-agent/Agentless 15.6pp gap (at SHA `5ce5888`).

aider: benchmark-runner broad `except` at `benchmark/benchmark.py:L666-L676` stores the exception as `{"exception": ...}` row; aggregation at `:L502-L512` divides by `len(rows)`, so exception rows contribute 1 to the denominator and 0 to the numerator without surfacing the infra-flake count.

Mind2Web: `IndexError` on out-of-bounds choice index caught at `src/action_prediction/metric.py:L203-L209` (`logger.info`-only), falls through to score-0 + kept in denominator (P1); `src/action_prediction/dataloader.py:L277` applies a train-only positive-candidate filter, creating a train/eval data-distribution asymmetry (P7).

open-r1: E2B broad `except` at `src/open_r1/rewards/code_providers.py:L107-L112` coerces sandbox exceptions to `[0.0]*len(scripts)` rewards with print-only surface; MorphProvider identical pattern at `:L248-L249`; IOI/Codeforces paths enter TRL's `None`→`nansum` pipeline via `:L396-L397, L449-L450`.

ragas: Pydantic parser fallback at `ragas/prompt/pydantic_prompt.py:L315-L334` (self-fix loop raises `RagasOutputParserException`); executor broad `except` returns `float("nan")` at

ragas/executor.py:L64-L86; safe_nanmean at ragas/utils.py:L46-L55 silently shrinks the denominator by the NaN count across faithfulness, answer-relevancy, and context-precision metrics.

MTEB: leaderboard aggregate filters out models with any NaN task score at mteb/leaderboard/_create_table.py:L136-L149 (P4 selection bias); per-language aggregates at mteb/results/task_result.py:L526-L563 compute over produced scores with no attempt-vs-score coverage counter.

promptfoo (TypeScript): any LLM-judge parse failure coerced to {pass: false, score: 0} at src/matchers/llmGrading.ts:L499, L515 (P6); fail helpers at src/matchers/shared.ts:L25-L38 return the same hardcoded score indistinguishably from legitimate zeros; provider-error path at src/evaluator.ts:L1046-L1051 produces score: 0 without a providerErrored flag.

Score-1 repositories (9, one-liner each).

SciCode: partial-submission aggregation at eval/scripts/test_generated_code.py:L125-L127.

inspect_ai: fail_on_error=True by default (_eval/task/error.py:L26), but fail_on_error=False opt-in degrades silently (scorer/_metrics/accuracy.py:L33).

ScienceAgentBench: swallows EvaluationError/BuildImageError/Exception (evaluation/harness/run_evaluation.py:L118-L136); discards timed_out flag (:L113); GPT-4o judge regex-miss→score 0 at gpt4_visual_judge.py:L70-L72.

android_world: np.nan + DataFrame.mean asymmetric drop at task_evals/task_eval.py:L249-L259 and android_world/suite_utils.py:L544-L558, L681-L698.

OSWorld (harness): bare-except "Time limit exceeded" at run.py:L205-L218; env.evaluate() FileNotFoundError→0 at desktop_env/desktop_env.py:L485-L487; Verified-subset non-comparability disclaimer at README.md:L36.

simple-evals (non-SimpleQA subset): honest grading on multi-choice, regex miss→wrong on GPQA/MMLU (see Appendix **??**, Smoking-Gun #2).

HELM: Stat.add(None) silently skips None results at src/helm/benchmark/metrics/statistic.py:L35; judge parse returns None at gpt4_audio_critique_metrics.py:L70-L72 and is absorbed by the Stat.add(None) skip. The per-scenario attrition rate is recoverable from the full output JSON but is not in the headline report (at SHA 83bde5c).

BIG-bench: aggregate metric computation at bigbench/api/results.py:L560-L576 uses statistics.mean(d[m] for d in per_task if m in d), silently shifting the denominator by the count of tasks that did not produce metric m. Per-task results are preserved in raw output so the shift is recoverable, but the headline mean does not emit a coverage counter (at SHA 092b196).

deepeval: JSONDecodeError on LLM-judge output re-raised as ValueError at deepeval/utils.py:L405-L413 and caught broadly by the test-runner; safe_a_measure at deepeval/progress_context.py:L299-L305 catches any metric-evaluation exception. Per-test failure messages are surfaced but parse-failure rates are not aggregated into a headline counter (at SHA f917b5a).

A.6 Verification

Every source-level claim in this appendix is tied to a pinned commit SHA and a line-level permalink in the supplementary survey document. Claims were retained only when the cited lines directly supported the stated failure-handling behaviour. The audit therefore fixes both the repository version and the exact code location being asserted, making each entry independently checkable without relying on moving default branches.

A.7 Pinned commit SHAs

Every permalink in this appendix resolves against the following pinned commits. Repositories are grouped by survey stratum; full 40-character SHAs and line-level permalinks are provided in the supplementary survey document.

SWE-bench-family harnesses and agent scaffolds. SWE-bench `f7bbbb2`; SWE-agent `0f4f3bb`; mini-swe-agent `bc85a45`; live-swe-agent `8d7dd86`; SWE-bench_Pro-os `0c64e26`; SWE-smith `9b74ac0`; Agentless `5ce5888`; aider `f09d7065`.

RL and agentic-RL training frameworks. TRL `88826fd`; verl `6eeb571`; OpenRLHF `64c1cc4`; rllm `ea623cc`; slime `5b688aa`; ART `679b236`; Agent-R1 `38bdfc1`; RAGEN `20daedc`; MARTI `a2fe2c7`; MATPO `3c41d62`; RL-Factory `d0abc1d`; Trinity-RFT `9051d63`; ms-swift `c4902f3`; verifiers `e27633b`; open-r1 `1416fa0`.

General evaluation harnesses and leaderboard infrastructure. openai/evals `8eac7a7`; simple-evals `ee3b031`; SciCode `e3158ea`; lm-evaluation-harness `c1c4bea`; lighteval `10b9104`; inspect_ai `36231d6`; FastChat `587d5cf`; HELM `83bde5c`; ragas `298b6827`; deepeval `f917b5a`; MTEB `9363ea75`; promptfoo `3dc5843`; BIG-bench `092b196`.

Web, GUI, mobile, scientific-agent, and ML-engineering benchmarks. WebArena `dce0468`; VisualWebArena `89f5af2`; OSWorld `c7e54d2`; android_world `d9c569f`; Mind2Web `33bd95c`; MLE-bench `2451bcb`; MLAgentBench `5d71205`; ScienceAgentBench `1cf1375`.

Function-calling, multi-agent, and general agent-scaffold repositories. gorilla (BFCL) `6ea5797`; ToolBench `d56fdd8`; AgentBench `d1e4a10`; AgentBoard `bb7255e`; tau-bench `59a200c`; Self-rewarding-reasoning-LLM `372bea9`. The out-of-scope observation (OpenHands, documented as a scope-filter exemplar) is pinned at `3b17f27`.

A.8 Coverage snapshot

The final sample contains 50 in-scope repositories and two documented exclusions. It spans SWE-bench-family harnesses, agent scaffolds, RLHF trainers, agentic-RL trainers, multi-agent RL, web/GUI agents, mobile agents, ML-engineering benchmarks, scientific-agent benchmarks, function-calling benchmarks, general eval frameworks, LLM-judge infrastructure, autonomous coding assistants, RAG-eval frameworks, embedding/retrieval benchmarks, red-team/prompt-eval tooling, and rejection-sampling SFT pipelines. The severity distribution is 11 score-3 entries, 31 score-2 entries, 9 score-1 entries, and 0 score-0 entries. Because `simple-evals` is scored separately for SimpleQA and non-SimpleQA behaviour, these 51 entries correspond to 50 unique repositories. Seven silent-drop patterns covering reward coercion, rollout filtering, bypassed saves, survivor denominators, partial-trajectory drops, LLM-judge parse→fixed score, and outcome-as-ground-truth SFT filters. The audit contains 50 pinned SHAs and approximately 160 verified source citations. Every code claim in this appendix is a commit-pinned two-click audit: if any claim fails to hold up at its permalink, that is a bug in this appendix, not a vague disagreement about code that has since moved — the SHAs fix the audit target.

B 37-pair Variance Catalogue

This appendix enumerates the 37 cross-harness variance pairs sampled in Table **??**, organised by metric family: 22 task-success pairs (§**??**), 9 cost/token pairs (§**??**), and 6 latency/timing pairs (§**??**). Four task-success benchmarks are then examined in *smoking-gun* detail (§**??**) — same dataset, multiple in-survey implementations, different numbers on identical workloads. Training-side evidence beyond the rollout interface (Dr. GRPO, DAPO, TRL's five GRPO variants, verl #2165) is consolidated in §**??**. A de-duplication log (§**??**) records which family each pair is counted in, and §**??** catalogues leads that did not meet the evidence bar.

B.1 Evidence hierarchy

Every pair is graded against the same four-tier evidence hierarchy, uniform across the three metric families:

1. **Tier 1 — strongest.** Same model and workload; published numbers disagree across two harnesses. (Direct empirical evidence.)
2. **Tier 2.** Explicit vendor or framework documentation acknowledging the convention difference between harnesses. (Vendor-admission-equivalent.)

3. **Tier 3.** Code-level convention divergence with no cross-framework normalization, published under the same metric name. (Structural divergence visible at source.)
4. **Tier 4.** Maintainer issue-tracker statement that the convention differs from another framework. (Maintainer-acknowledged divergence.)

Anything weaker is demoted to near-misses (§**??**). Every pair below cites either a numeric delta, a vendor acknowledgement, a code-level convention divergence, or a maintainer statement.

B.2 Task-success pairs (22)

Same model weights, same benchmark split, different published accuracy depending on which harness computed the aggregate.

Table 4: 22 task-success variance pairs. Δ is the gap between published numbers on an identical workload. "post-mortem" refers to the HuggingFace Open LLM Leaderboard post-mortem (**?**).

#	Model	Benchmark	Harness A	Harness B	Δ	Source
T1	LLaMA-65B	MMLU 5-shot	Berkeley orig.	lm-eval-harness	14.9pp	post-mortem
T2	LLaMA-30B	MMLU 5-shot	Berkeley orig.	lm-eval-harness	12.6pp	post-mortem
T3	Falcon-40B	MMLU 5-shot	Berkeley orig.	lm-eval-harness	10.5pp	post-mortem
T4	Llama-3.1-70B-Inst	MMLU-Pro	Meta self (CoT)	OLL v2 / lighteval	∼18.5pp	Meta card vs OLL
T5	Qwen2.5-72B	MMLU-Pro	Self-report	OLL v2	∼22pp	Qwen docs vs OLL
T6	GPT-4o (24-05-13)	SWE-b. Verified	SWE-agent	Agentless 1.5	15.6pp	swe-b. subs. S3
T7	Claude 3.5 Sonnet	SWE-b. Verified	Anthropic Tools	OpenHands CA 2.1	4.0pp	SWE-b. leaderb.
T8–T13	Various	SWE-b. Lite/Ver.	Various	Various	3.4–19.6pp	SWE-b. leaderb.
T14	Mistral-7B	MMLU (prompt)	Template A	Template B	up to 24.6pp	Biderman et al.
T15	Mixtral-8x7B	MMLU (prompt)	Template A	Template B	up to 24.6pp	Biderman et al.
T16	Mistral-7B	ARC (prompt)	Template A	Template B	up to 24.6pp	Biderman et al.
T17	Llama-3.1-70B	MATH	Meta (full test)	Leaderboard (500)	label collision	Meta vs leaderb.
T18	Claude 3.5 Sonnet	MATH	"MATH"	"MATH"	label overload	multi-source
T19	Llama-3.1-70B	GPQA	simple-evals	lm-eval-harness	vendor admis.	NVIDIA NeMo
T20	Llama-3.1-70B	GPQA	Self-report	Leaderboard	cross-harness	Meta + leaderb.
T21	Llama-3.1-70B	Various	Meta model card	Meta `eval_details.md`	micro vs macro	Meta own docs
T22	Llama-3.1-70B	Various	`acc_char`	full-string match	convention	Meta own docs

T1 (LLaMA-65B MMLU 14.9pp) is the flagship single pair — identical weights, different published score, explicitly discussed in a HuggingFace post-mortem. T6 (GPT-4o SWE-bench Verified 15.6pp) is the target of the reconciliation experiment in Sec. **??**: trajectories for both runs are in the public SWE-bench submissions S3 bucket, making the reconciliation tractable. T14–T16 (Biderman et al.) are not cross-harness pairs but within-harness prompt-template pairs, included because the 24.6pp gap is the strongest published evidence that the instrument-level variance is at least as large as the signals it purports to measure.

B.3 Cost / token pairs (9)

Same model, same workload, different published token counts or dollar costs depending on which tool applies which convention. Table **??** summarises; for numerical examples and code-level references see the supplementary survey document and the cost-latency divergence report in supplementary materials.

Table 5: 9 cost / token-accounting variance pairs. Same model, same workload, different published numbers.

#	Setup	Harness A	Harness B	Δ
C1	Anthropic w/ `cache_control`	OTel `input_tokens` (inclusive)	Anthropic-native (exclusive)	2.0× inflation (260,421 vs 130,213)
C2	Claude + cache_control	LiteLLM cost estimator	Anthropic Console billing	+68% overestimate ($0.091 vs $0.054)
C3	Claude (streaming) + cache-hit	LangChain.js accumulator	single `message_delta`	exactly 2× cache_read
C4	SWE-b. V. retry-firing task	SWE-agent (every attempt)	mini-swe-agent (last attempt)	30–100% per-task overstatement
C5	Anthropic/OpenAI prompt-cache	inspect_ai (cache-aware, tiered)	7 peer harnesses (no cache parse)	up to 90% on cached portion
C6	Aider leaderboards (same model date)	Aider Edit	Aider Refactor / Polyglot	$0 → $14.41 (same model)
C7	Same rollout batch, GRPO training	verl `loss_agg_mode=token-mean`	verl `seq-mean-token-sum`	different gradient magnitudes
C8	Cross-vendor "tokens used"	tiktoken (`o200k_base`)	Anthropic BPE	∼10% systematic
C9	OpenAI reasoning models (o1/o3)	harness reading `completion_tokens` only	harness reading `..details.reasoning_tokens`	silent under-report

Code-level sources. **C1:** OpenTelemetry PR #3163; Langfuse issue #12306. **C2:** BerriAI/litellm issue #9812. **C3:** langchainjs issue #10249. **C4:** `sweagent/agent/models.py` L744–L838; `minisweagent/models/litellm_model.py` L80–L93. **C5:** `inspect_ai/model/_openai.py` L776–L782; `inspect_ai/model/_model.py` L2085–L2091; `inspect_ai/model/_providers/anthropic.py` L1137–L1169. **C6:** `edit_leaderboard.yml`, `refactor_leaderboard.yml`, `polyglot_leaderboard.yml` in `Aider-AI/aider`. **C7:** `verl/trainer/ppo/core_algos.py` L1168–L1195. **C8:** tiktoken and Anthropic tokenizer documentation; openai/tiktoken issue #474. **C9:** OpenAI API reference for reasoning-model `usage.completion_tokens_details`.

C1 is the flagship: OpenTelemetry PR #3163 formalises the Anthropic/OpenAI/Vertex convention disagreement at the observability-middleware layer, and Langfuse issue #12306 exhibits the numerical 260,421-vs-130,213 gap on a single request. C6 is the flagship intra-vendor disagreement: the same organisation's three leaderboards (Edit, Refactor, Polyglot) publish $0, $8.46, and $14.41 for the same model on comparable tasks, with no convention statement in any of the three YAMLs.

B.4 Latency / timing pairs (6)

Same workload, same hardware, different published wall-clock times depending on which harness's timer wraps which phase.

Table 6: 6 latency / wall-clock variance pairs. Clock-start and clock-stop differ across harnesses that log the same metric name.

#	Setup	Harness A	Harness B	Δ
L1	SWE-b. V. (same task)	Docker backend	Modal backend	systematic Modal inflation
L2	OSWorld (same task)	`lib_run_single.py`	`scripts/python/run_maestro.py`	≥30 s (hardcoded sleep) + 5–15 s VM start
L3	GRPO step timer	TRL `training_step` (grad+opt only)	verl `marked_timer("step")` (full epoch)	up to 5× on rollout-dominated runs
L4	Qwen2.5-7B GSM8K-GRPO, same HW	TRL	OpenRLHF	3.13× (5,189 s vs 1,657 s)
L5	GRPO throughput	verl `perf/throughput` (excl. padding)	TRL effective-token throughput	different numerators on same HW
L6	inspect_ai timers	`sample_working_time`	`sample_waiting_time`	ambiguity declared in-tool

Code-level sources. **L1:** `swebench/harness/docker_utils.py` L203–L217 vs `swebench/harness/modal_eval/run_evaluation_modal.py` L307–L319. **L2:** `lib_run_single.py` #L616 vs `scripts/python/run_maestro.py` #L351. **L3:** `trl/trainer/grpo_trainer.py` L1111–L1120; `verl/trainer/ppo/metric_utils.py` L313–L346. **L4:** arXiv:2501.03262 Table 4; EMNLP-demos 2025. **L5:** `verl/trainer/ppo/metric_utils.py` L337–L345. **L6:** `inspect_ai/_util/working.py`.

L4 is the flagship: published, peer-reviewed, same hardware, same model, same algorithm — a 3.13× wall-clock gap attributable to framework-level dispatch rather than algorithm difference. L1 and L2 are in-repository disagreements: both backends in SWE-bench Verified and both harnesses in OSWorld ship under the same repo at the same commit, reporting the same metric name, with no statement that the two are not comparable.

B.5 Smoking-gun benchmarks: same dataset, multiple in-survey implementations

Four benchmarks are implemented in 3–4 surveyed repositories each. Below, each implementation's grading site and failure-handling semantics are read off at the pinned SHAs (§**??**). For each benchmark, a single realistic failure mode produces a different headline number in each implementation — and none of the implementations emits a counter disclosing which regime it fell into.

Smoking-Gun #1: SWE-bench (4 implementations). *Official SWE-bench* grades via `bad_codes` = {APPLY_PATCH_FAIL, RESET_FAILED, TESTS_ERROR, TESTS_TIMEOUT} (`swebench/harness/grading.py:L61-L70`; `constants/__init__.py:L80-L89`) with empty patches pre-filtered from submission (`docs/faq.md:L39`). Container timeout stays in the denominator as a fail; empty-patch instances are removed entirely. *PrimeIntellect verifiers* reads the same `bad_codes` set but wraps `test_spec` fetching in a 5× tenacity retry; any uncaught exception flows through `rubric.py:L144-L158` to `0.0` and enters the group-mean advantage baseline — a container timeout can bias the policy gradient for the other rollouts in its group rather than merely decrementing the numerator. *SWE-agent* delegates grading to `sb-cli submit` (`sweagent/run/hooks/swe_bench_evaluate.py:L42-L55`); the pre-submission `unlink` at `run_batch.py:L397-L401` removes crashed or cost-killed instances from `preds.json` entirely, so server-side grading sees a smaller denominator instead of a failed row. *mini-swe-agent* has no local grading; its `finally: save()` at `benchmarks/swebench.py:L171` is bypassed by `KeyboardInterrupt` (closed issue #329), also producing an absent-from-submission pattern. On one realistic mid-test Docker timeout, the four harnesses report: Official $0/N$, verifiers $1/N$ (possible retry recovery), SWE-agent $0/(N-1)$, mini-swe-agent $0/(N-1)$. Inspect AI has *no SWE-bench bridge* at the pinned SHA.

Smoking-Gun #2: GPQA (3 implementations). *simple-evals* at `gpqa_eval.py:L59` uses `re.search(ANSWER_PATTERN_MULTICHOICE, response_text)` and `extracted_answer = match.group(1) if match else None`; a regex miss — which a hedging response triggers — is scored 0 and stays in the denominator. *lm-evaluation-harness* uses log-likelihood of the gold-choice token over the various `gpqa_*_zeroshot.yaml` / `_n_shot.yaml` / `_cot_*.yaml` tasks; the model does not generate free text in the scored branch, so hedging is immune, but silent left-truncation at `huggingface.py:L1360-L1368` still affects long contexts. *lighteval* scores via the standard pipeline at `tasks/tasks/gpqa.py` with aggregation through `info_loggers.py:L326-L400`; API retry-exhaust or content-filter yields an empty `LitellmModelResponse()` (`litellm_model.py:L243, L254`), scored as wrong via `metrics_sample.py:L151-L152` (`if not pred: return 0`). On a single hedging model output: simple-evals 0, lm-eval-harness likely correct, lighteval wrong — three different headline accuracies from the same model reasoning. Inspect AI has *no GPQA task file* at the pinned SHA.

Smoking-Gun #3: MATH / MATH-500 (3 implementations). *simple-evals* at `math_eval.py:L55` calls `check_equality(self.equality_checker, row["Answer"], extracted_answer)`, which itself calls the LLM equality checker; an API failure raises (dropping the sample) or is coerced to 0 by the outer handler. *lighteval* uses majority@n with a symbolic `math_normalizer` over `tasks/tasks/math_500.py` and `math.py`; normalizer exceptions are caught upstream and scored wrong at `metrics_sample.py:L151-L152`. *PrimeIntellect verifiers* routes reward functions through `rubric.py:L144-L158`; exceptions become `0.0` and enter the group advantage baseline. On a LaTeX edge case such as `\frac{1}{2}` versus `0.5`: simple-evals's

LLM judge may call these equal, lighteval's normalizer may not, and verifiers's reward exception enters training as an errored zero — same completion, three different outcomes.

Smoking-Gun #4: MMLU (3 implementations). *simple-evals* (`mmlu_eval.py`) extracts answers by regex on generated text — same miss-is-wrong pattern as GPQA. *lm-evaluation-harness* (`lm_eval/tasks/mmlu`: 12+ variants spanning main, redux, pro, flan, etc.) uses log-likelihood over A/B/C/D tokens with `acc`/`acc_norm`; silent prompt truncation at `huggingface.py:L1360-L1368` logs the truncated-prompt sample as a complete result. *lighteval* uses exact-match on the choice label (`tasks/tasks/mmlu.py`), with left-truncation at `vllm_model.py:L374-L397` warning but recording no `truncated_tokens_count`. On a long-context MMLU-Pro prompt that overflows a 4k model: simple-evals generates a regex answer against the truncated prompt the model saw (right or wrong depending on the content); lm-eval-harness left-truncates and logs as complete (silently wrong); lighteval left-truncates, records no truncation counter, and logs as complete (silently wrong, with no recoverable audit trail). Three implementations, three behaviours, no `n_prompt_truncated` counter in any output.

B.6 Training-side evidence beyond the rollout interface

Operator disagreement extends into the gradient-signal computation at training time. The paper cites these as published findings rather than original contributions.

Dr. GRPO (**?**) corrects only the loss-aggregation convention — the length-normalization bias documented in "all popular open-source PPO implementations" — and reports +7.3 points on AIME 2024 vs SimpleRL-Zero-7B and +15.7 points on AIME 2024 vs Prime-Zero-7B, with no change to data, model, or algorithm. **DAPO** (**?**) is a catalogue of framework-level defaults (zero-std group filter, token-vs-sequence loss aggregation, decoupled clip ranges) that had to be flipped to match DeepSeek-R1; the paper reports 50 points on AIME 2024 for Qwen2.5-32B on the defaults-flipped run. All three defaults are silently different across TRL, verl, and OpenRLHF. **TRL's own library** ships five mutually-incompatible GRPO variants — `grpo`, `dr_grpo`, `dapo`, `bnpo`, and `importance_sampling_level`∈{`token`, `sequence`} — in a single config file (`trl/trl/trainer/grpo_config.py:L228-L243, L668-L677`); two papers reporting "we trained with GRPO on task X" are structurally non-comparable without specifying which variant. **verl #2165** (**?**) is a tokenization-channel divergence on Qwen3-4B GRPO where the training-side FSDP tokenizer inserts `<think>\n\n</think>` into assistant turns that the rollout engine (vLLM / SGLang) never saw, shifting the assistant-turn token mask by tens of tokens across ∼40k-token conversations. The finding is reproduced by six independent users on the thread, upstream-confirmed in QwenLM/Qwen3 #1826 and Qwen/Qwen3-1.7B HF discussion #9, and guarded separately by verl's own `tokenization_sanity_check_mode` (distinct from the dtype guard in discussion #5984) — proving that tokenization-class mismatch exists as a separate failure mode even when all upstream dtypes match.

Silent rollout-dropping mechanisms (beyond the 50-repo audit). verl #1170 (rollouts silently return empty strings, training proceeds); OpenRLHF #1108 (reward curves diverge between vLLM 0.8.1 and 0.8.3 on the same seed); verl discussion #5984 (tool built specifically to detect silent per-token log-prob divergence between training FSDP and vLLM rollouts); ms-swift #9096 (gemma-4 rollouts return garbage, training continues). **Numerical-layer evidence.** BF16 backends have been documented to produce divergent log-probs between training and rollout engines even with identical weights ("Defeating the Training-Inference Mismatch via FP16," arXiv:2510.26788); the vLLM engineering blog documents separate backends breaking the on-policy assumption of policy-gradient methods; THUDM's `slime` ships a `train_infer_mismatch_helper` because divergence is expected.

B.7 De-duplication log

Several findings could plausibly appear in more than one family. For audit transparency we record where each is counted and why:

- **Dr. GRPO**, **DAPO**, **TRL's five GRPO variants**: counted in §**??** (training-side); not in §**??** despite touching per-step token accounting. Rationale: the load-bearing claim is about gradient signal, not token counts.

- **verl #2165** (Qwen3 tokenization-channel divergence): counted in §**??** (training-side); not in §**??** despite involving silent per-rollout mismatch. Rationale: Appendix **??** audits *failure-handling* patterns; verl #2165 is a *convention-disagreement* finding, structurally closer to Dr. GRPO than to `try/except/return None`.
- **TRL vs OpenRLHF 3.13× wall-clock** (L4): counted in §**??**; not in §**??** despite the training-pipeline context. The headline delta is wall-clock, not gradient signal.
- **verl** `loss_agg_mode` **variants** (C7): counted in §**??**; not in §**??** despite affecting per-step timing.
- **verl throughput metric** (L5): counted in §**??**; not in §**??** despite involving token counting.
- **inspect_ai** `sample_working_time` **vs** `sample_waiting_time` (L6): counted in §**??**; not in §**??** despite inspect_ai's role as the positive control for failure-handling.

No finding is counted twice across the 37 pairs.

B.8 Near misses

Leads that did not meet the evidence bar (§**??**) but warrant human follow-up.

Cost / tokens. Anthropic `/v1/messages/count_tokens` versus tiktoken estimate — Anthropic publishes a dedicated counting endpoint, community harnesses routinely substitute tiktoken for speed, the endpoint returns different numbers on the same input; documented but no published benchmark pair with materially different totals attributable to this choice. Anthropic `/cost` underreport vs dashboard (anthropics/claude-code #1063): intra-vendor, not cross-harness. HuggingFace Open LLM Leaderboard v1→v2 transition (including the lighteval vs lm-eval-harness switch): full methodology change with no explicit "efficiency numbers not comparable across versions" statement.

Latency. HELM `efficiency.json` publishes per-model latency/throughput but is frozen at pre-2023 models with no modern cross-comparison. OpenAI vs Azure OpenAI latency differences on the same model are documented but with no published benchmark pair.

Task success. Anecdotal "our replication differs from the original paper" issue-tracker threads that do not cite a specific convention difference.

C Projection Operators to Community Trace Shapes

This appendix defines each projection $\pi : \tau_E \to (T_\pi, D_\pi)$ whose structural preservation properties appear in Table **??**. For each we give: (i) the target trace format as a formal object; (ii) pseudocode for the projection function operating on $\tau_E = (N, E, C, A, M)$; (iii) the drops manifest D_π as a list of typed erasure records; (iv) a characterisation of which substrate structure the projection preserves. Reference implementations are in the released codebase; the pseudocode here is the specification, not the implementation. Each drops manifest record is a tuple $(\textit{erasure_type}, \textit{element_ref}, \textit{metric_class})$ per §**??**.

Figure **??** gives the visual summary before the definitions: one SWE-Bench Verified rollout τ_E rendered through all six projections, with each projection's drops manifest overlaid.

Preservation, operationally. We use "projection" informally — a function that loses information. A behavioural quantity $\mu : \tau_E \to \mathbb{R}$ is *π-preserved* iff there exists a computable g such that $g(\pi(\tau)) = \mu(\tau)$ for all τ; that is, μ factors through π. Table **??** records three cases per (projection, quantity) pair: (i) **preserved** (✓): such a g exists given T_π's canonical schema alone; (ii) **erased** (✗): no such g exists because T_π lacks the relevant channel; (iii) **partial** ($\sim$): g exists on a subset of τ_E but not universally, admitting a numeric preservation fraction $\rho_{\pi,\mu} \in (0, 1)$ whose computation requires running π against a trajectory corpus and is beyond the scope of this paper. Each cell is therefore a concrete question ("does g exist given T_π's schema?") rather than a qualitative judgement.

Table **??** reads off the projection definitions below. The pattern is consistent: the five community projections each preserve their own native quantity on the diagonal and (mostly) erase the others; π_{step} carries no structural channel beyond the token-indexed step sequence and so erases every non-self-community question; $\pi_{\text{json-log}}$ preserves every quantity by construction but is not a training target

Figure C.1: One SWE-Bench Verified trajectory, six representations. *Centre: substrate state τ_E for one rollout, rendered with containment tree (vertical), dependency DAG (horizontal edges), reasoning-log events (timeline), and typed mutations (marked). Five surrounding panels: the same trajectory under each projection below. Drops manifest records overlaid on each panel, colour-coded by erasure type (concurrency collapse, cancellation erasure, edge deletion, containment flattening, reasoning truncation, attribution loss). π_{step} and $\pi_{per\text{-}agent}$ each show large shaded regions (structural erasure); $\pi_{call\text{-}tree}$ serialises concurrent siblings into an ordering; π_{macro} collapses sub-tree reasoning into aggregate effects; $\pi_{json\text{-}log}$ matches the substrate up to schema encoding.*

Figure 2: One SWE-Bench Verified trajectory rendered through all six representations, with drops manifests overlaid.

Table 7: Structural preservation matrix. Rows: the six projections defined below plus the raw substrate τ_E. Columns: the six community-native behavioural quantities of §**??** (**Step-ret**: step-indexed return under episode mask, long-horizon LLM-agentic RL (**??**); **Opt-term**: option termination frequency, HRL/macro-action (**?**); **Call-dep**: call-tree depth distribution, recursive LMs (**?**); **Sib-dist**: per-agent sibling embedding distance, fixed-role MAS (**?**); **MCTS-ent**: node visit entropy across frontier, MCTS-based training (**??**); **Async-dp**: dispatch-and-wait rate, async MAS). ✓ = preserved under π; ✗ = T_π has no channel that could carry the elements contributing to the quantity; $\sim$ = partially preserved. Cells follow from the projection pseudocode in §§**??**–**??**.

Projection	**Step-ret**	**Opt-term**	**Call-dep**	**Sib-dist**	**MCTS-ent**	**Async-dp**
π_{step} (long-horizon RL)	✓	✗	✗	✗	✗	✗
π_{macro} (hierarchical)	$\sim$	✓	✓	✗	✗	$\sim$
$\pi_{\text{call-tree}}$ (recursive LM)	$\sim$	$\sim$	✓	$\sim$	✗	✗
$\pi_{\text{per-agent}}$ (fixed-role MAS)	$\sim$	✗	✗	✓	✗	$\sim$
π_{mcts} (search-tree)	✗	✗	$\sim$	✗	✓	✗
$\pi_{\text{json-log}}$ (production ref.)	✓	✓	✓	✓	✓	✓
τ_E (Ergon raw)	✓	✓	✓	✓	✓	✓

for any of the five communities. The off-diagonal $\sim$ cells (e.g., $\pi_{\text{call-tree}}\times$ Sib-dist) indicate cases where T_π carries some but not all of the channel μ requires: $\pi_{\text{call-tree}}$ preserves sibling *relations* but not the reasoning-log content embeddings MAST's role-differentiation metric computes over.

A methodological note: Table **??** asks whether μ factors through π, not whether any proxy for μ does. A weaker proxy claim — say, "fraction of episodes terminated before the environment terminates" as a surrogate for option termination under π_{step} — may survive erasure of the canonical quantity. Whether such a proxy counts as the same claim is a judgement the framework does not adjudicate; it establishes only that the canonical μ, as defined over τ_E, does not factor through π.

C.1 π_{step}: step-indexed (o_t, a_t, r_t) tuples

Target format. A token-indexed flat prompt-response tensor pair, matching the `DataProto` schema (**?**) consumed by VERL, OpenRLHF, and TRL: $(\texttt{prompts}, \texttt{responses}, \texttt{attention_mask}, \texttt{response_mask}, \texttt{token_level_scores}, \texttt{advantages})$, where `token_level_scores` is a per-token reward tensor of shape [batch, response_length]. Equivalently, a sequence $T_{\text{step}} = [(o_1, a_1, r_1), (o_2, a_2, r_2), \ldots]$ where each step corresponds to a token position and $r_t \in \mathbb{R}$ is its assigned reward. Multi-turn interactions are handled by concatenating turn tokens into one flat response per rollout; no per-turn identity is preserved in the tensor shape.

Projection pseudocode.

```
def pi_step(tau_E):
```

```
    N, E, C, A, M = tau_E
    T_step = []
    D = [] # drops manifest
    # Walk M in sequence order; emit one step per
    # (assistant_text | tool_call -> tool_result) pair
    # from the root node's reasoning log only.
    root = find_root(N)
    events = [e for e in C if e.node_id == root.id]
    for e in events_by_sequence(events):
        if e.kind == "assistant_text" or e.kind == "tool_call":
            a_t = e.payload
            o_t = next_observation_after(e, C)
            r_t = reward_for(e, A) # from annotation store
            T_step.append((o_t, a_t, r_t))
    # Record drops
    for n in N:
        if n.id != root.id:
            D.append(("containment_flattening", n.id, "depth"))
    for edge in E:
        D.append(("edge_deletion", edge.id, "tree_likeness"))
    for m in M:
        if m.kind in ("node.added", "node.status_changed")
            and m.node_id != root.id:
            D.append(("cancellation_erasure"
                      if "cancel" in m.payload else
                      "attribution_loss",
                      m.id, "late_cancel"))
    for e in C:
        if e.node_id != root.id:
            D.append(("reasoning_truncation", e.id, "role_diff"))
    return T_step, D
```

Preserved. The root worker's linear sequence of assistant text and tool interactions with its own environment.

Dropped. Everything else: the containment tree beyond the root, all dependency edges, all cross-worker causal structure, all cancellation events as typed policy actions (they appear, if at all, only as the absence of subsequent children), per-worker identity beyond the root, and the reasoning logs of all non-root workers.

C.2 $\pi_{\text{per-agent}}$: per-worker streams with partner annotations

Target format. A mapping from worker identifier to a per-worker stream: $T_{\text{per-agent}} = \{w_i \mapsto \sigma_i\}_{i=1}^{k}$, where each σ_i is a per-worker stream of observations and actions for worker w_i under a topology declared before the rollout, optionally annotated with role tags or partner-type beliefs. This shape covers fixed-role multi-agent LLM systems, MARL with a centralised critic, proposer/critic debate setups, and — as a special case of coordination under partner uncertainty — ad hoc teamwork (**??**). Cross-stream references are limited to dependency edges between streams, which the format does not natively carry.

Projection pseudocode.

```
def pi_per_agent(tau_E):
    N, E, C, A, M = tau_E
    workers = group_by(N, key=lambda n: n.assigned_worker_key)
    T_per_agent = {}
    D = []
    for w, nodes in workers.items():
        stream = []
        events = [e for n in nodes for e in C if e.node_id == n.id]
        for e in events_by_sequence(events):
            if e.kind in ("assistant_text", "tool_call",
                          "tool_result"):
                stream.append(event_to_stream_record(e))
        T_per_agent[w] = stream
```

```
    # Drops
    for n in N:
        if n.parent_node_id:
            D.append(("containment_flattening", n.id, "depth"))
    for edge in E:
        src_w = worker_of(edge.src, N)
        dst_w = worker_of(edge.dst, N)
        if src_w != dst_w:
            D.append(("edge_deletion", edge.id, "tree_likeness"))
    for m in M:
        if m.kind == "node.status_changed"
            and m.payload.status == "cancelled":
            D.append(("cancellation_erasure", m.id, "late_cancel"))
    return T_per_agent, D
```

Preserved. Per-agent trajectory streams indexed by agent identity, matching the shape MARTI (**?**) distributes to individual policy trainers. Partial concurrency information: a consumer comparing wall-clock timestamps across streams can detect overlapping activity. Partial role differentiation: embedding analysis across the per-agent streams remains possible, though the stream-identity boundary is now a projection artefact rather than a containment-tree fact.

Dropped. The format is agent-major rather than work-major: task-level structure — which subtask is shared across which agents, which agents are blocked on each other's dispatches, which branches were cancelled as typed events — is carried by the workflow class in systems like MARTI (Multi-Agent Debate, Mixture-of-Agents, Chain-of-Agents) and is not recoverable from the per-agent streams alone. Specifically: containment depth (the parent-child relation flattens to stream identity); cross-worker dependency edges; cancellation as a typed action (worker terminations appear only as stream endings); async dispatch-and-wait (the dispatch is in one stream, the result in another, with no expressible causal link).

C.3 $\pi_{\text{call-tree}}$: nested sub-LM call tree

Target format. A tree $T_{\text{call-tree}}$ whose nodes are sub-LM invocations and whose edges represent parent-calls-child. Each node carries its prompt, its response, and its list of (serialised) child invocations. Concurrent siblings are linearised into a canonical child ordering, typically by start-time.

Projection pseudocode.

```
def pi_call_tree(tau_E):
    N, E, C, A, M = tau_E
    def build(node):
        children = [n for n in N
                    if n.parent_node_id == node.id]
        # Linearise by start time; this is the concurrency erasure
        children.sort(key=lambda n: n.created_at)
        return {
            "prompt": assembled_prompt(node, C),
            "response": assembled_response(node, C),
            "children": [build(c) for c in children],
        }
    root = find_root(N)
    T_call_tree = build(root)
    D = []
    for edge in E:
        D.append(("edge_deletion", edge.id, "tree_likeness"))
    for n in N:
        sibs = siblings(n, N)
        concurrent_sibs = [s for s in sibs
                           if overlaps_in_time(s, n, M)]
        for s in concurrent_sibs:
            D.append(("concurrency_collapse", (n.id, s.id),
                      "width"))
    for m in M:
        if m.kind == "node.status_changed"
            and m.payload.status == "cancelled":
```

```
            D.append(("cancellation_erasure", m.id, "late_cancel"))
    return T_call_tree, D
```

Preserved. Containment depth (the tree structure is the format). Per-node reasoning. Parent-child attribution.

Dropped. Concurrency (siblings are serialised into a canonical ordering, reducing measured width to one). Dependency edges (the format has a tree, not a DAG). Cancellation as a typed action (appears only as truncated child subtrees). Async dispatch-and-wait (result is a return value, not an event with a sequence stamp distinct from its dispatch).

C.4 π_{macro}: hierarchical macro-action decomposition

Target format. An option-tagged state-action trajectory: each primitive step (s_t, a_t) carries an additional label ω_t identifying the currently-active option, plus a termination flag $\beta_t \in \{0, 1\}$ marking when the option ended and the meta-policy selected a new one (**?**). The training signal is the termination gradient plus the intra-option policy gradient; by construction the framework is single-active-option, so at each step exactly one ω_t is active.

Projection pseudocode.

```
def pi_macro(tau_E):
    N, E, C, A, M = tau_E
    root = find_root(N)
    T_macro = []
    for child in direct_children_in_time_order(root, N):
        macro = {
            "s_start": state_at_start(child, M),
            "s_end": state_at_end(child, M),
            "t_start": child.created_at,
            "t_end": child.completed_at,
            "effect": aggregate_subtree(child, N, A),
        }
        T_macro.append(macro)
    D = []
    for n in N:
        if n.parent_node_id and n.parent_node_id != root.id:
            D.append(("reasoning_truncation", n.id, "role_diff"))
    for edge in E:
        D.append(("edge_deletion", edge.id, "tree_likeness"))
    for m in M:
        if m.kind == "node.status_changed"
            and m.payload.status == "cancelled":
            subtree_descendant = is_descendant(m.node_id, root, N)
            if subtree_descendant:
                D.append(("cancellation_erasure", m.id,
                          "late_cancel"))
    return T_macro, D
```

Preserved. Intra-option state-action transitions (each primitive step (s_t, a_t) is retained with its option label ω_t). Option boundaries, marked by the termination flag β_t. Temporal extension of each option. Aggregate effect and wall-clock timestamps per option.

Dropped. Concurrent option execution: the framework is single-active-option by construction (**?**), so any concurrency in τ_E is serialised. Cross-option dependencies: the options framework has no concept of one option's output being an input to another. Cancellation as a typed action: termination is a learned probability $\beta_\omega(s)$ rather than an event the agent chose, so there is no record of *why* a subtree was abandoned. Sub-worker identity within an option's subtree.

C.5 π_{mcts}: search tree with visit and value statistics

Target format. A search tree $T_{\text{mcts}} = (V, E_{\text{tree}}, B)$ where each vertex $v \in V$ carries a tuple (s_v, a_v, N_v, Q_v) recording the state, the action taken from the parent, the visit count, and the backed-

up value estimate; E_{tree} is the parent-child relation over V; and B is the set of backup edges (r, v) recording which rollout r contributed its terminal return to the value estimate at v. This is the format produced by rStar-Math (**?**) and ReST-MCTS* (**?**), consumed by AlphaZero-style training with PUCT visit distributions as policy targets and backed-up returns as value targets.

Projection pseudocode.

```
def pi_mcts(tau_E):
    N, E, C, A, M = tau_E
    # Identify nodes whose containment subtree constitutes
    # an MCTS rollout (by annotation tag or worker key).
    rollouts = [n for n in N if is_mcts_rollout(n, A)]
    # Build search-tree vertices by aggregating across rollouts
    # that share a state prefix.
    V = {} # keyed by (s, a) pair
    backup_edges = []
    for r in rollouts:
        for step_node in traverse_rollout(r, N):
            s = state_at(step_node, M, A)
            a = action_taken(step_node, C)
            key = (s, a)
            if key not in V:
                V[key] = {"s": s, "a": a, "N_visits": 0,
                          "Q_value": 0.0}
            V[key]["N_visits"] += 1
            # Back up terminal return from this rollout
            G = terminal_return(r, A)
            V[key]["Q_value"] = (
                (V[key]["Q_value"] * (V[key]["N_visits"] - 1) + G)
                / V[key]["N_visits"]
            )
            backup_edges.append((r.id, key))
    # Parent-child edges in the search tree
    E_tree = build_tree_edges(V, rollouts)
    T_mcts = (V, E_tree, backup_edges)
    # Drops manifest
    D = []
    for n in N:
        if not is_mcts_rollout_member(n, A):
            # Any delegation structure outside the search recipe
            D.append(("attribution_loss", n.id, "depth"))
    for edge in E:
        if not is_tree_edge_in(edge, E_tree):
            D.append(("edge_deletion", edge.id, "tree_likeness"))
    for m in M:
        if m.kind == "node.status_changed"
            and m.payload.status == "cancelled":
            D.append(("cancellation_erasure", m.id, "late_cancel"))
    for e in C:
        if e.kind in ("assistant_text", "thinking")
            and not is_part_of_rollout_state(e, rollouts):
            D.append(("reasoning_truncation", e.id, "role_diff"))
    return T_mcts, D
```

Preserved. Containment depth along the search-tree spine. Per-node visit counts and value backups (the signal the MCTS training step consumes). The rollout-to-node backup edges that explain which trajectory's terminal return updated which vertex value.

Dropped. Concurrency beyond fixed-width (s, a)-keyed vertices (sibling rollouts that explore the same (s, a) merge into a single vertex). Delegation structure outside the search recipe (any node not part of an MCTS rollout, e.g., concurrent tool calls not on the search tree, is dropped). Cancellation as a typed event (a cancelled rollout contributes no backup edge but leaves no explicit cancellation record). Cross-rollout dependency edges (the format has a tree by construction). Per-node reasoning text beyond what defines the state.

C.6 $\pi_{\text{json-log}}$: production-orchestration JSON event log (reference projection)

Target format. An ordered sequence of JSON records $T_{\text{json-log}} = [r_1, r_2, \ldots]$ where each r_i is a record with fields $\{\textit{sequence}, \textit{timestamp}, \textit{kind}, \textit{node_ref}, \textit{parent_ref}, \textit{payload}\}$. The record set is a near-isomorphism of the mutation log M extended with assistant text events from C.

Projection pseudocode.

```
def pi_json_log(tau_E):
    N, E, C, A, M = tau_E
    records = []
    combined = list(M) + [e for e in C
                          if e.kind in ("assistant_text",
                                        "tool_call",
                                        "tool_result",
                                        "thinking")]
    for item in sorted(combined, key=lambda x: x.sequence):
        records.append({
            "sequence": item.sequence,
            "timestamp": item.timestamp,
            "kind": item.kind,
            "node_ref": getattr(item, "node_id", None),
            "parent_ref": parent_ref_of(item, N, E),
            "payload": item.payload,
        })
    D = []
    # Drops manifest is empty of structural erasures;
    # only schema-serialisation losses apply (e.g., binary blobs
    # in annotations truncated to base64, if policy dictates).
    return records, D
```

Preserved. All structural mutations and all typed reasoning events, in sequence order, with full parent references and timestamps. The format is a near-isomorphism of (M, C).

Dropped. No structural erasure. The limitation of $\pi_{\text{json-log}}$ relative to τ_E is not lost expressiveness but the absence of (i) a formal projection interface to other community formats for cross-community analysis, and (ii) the drops manifest abstraction itself, so that downstream consumers of a JSON event log that re-encode it into their own format do so without explicit enumeration of what is lost. Ergon supplies both.

C.7 Adding a further projection

A new community trace format is added by writing one projection function $\pi_{\text{new}} : \tau_E \to (T_{\text{new}}, D_{\text{new}})$ in the same pattern as above. The substrate τ_E does not change; the drops manifest schema does not change; the structural preservation matrix gains one row. Declarative planning (**?**), embodied agent traces, and world-model-based planning are candidate further targets, which we leave to future work.

D Ergon System Details

D.1 Rollout-card format specification

The rollout-card format of §**??** is a medium-independent bundle: any backend preserving the row semantics below can emit and consume valid cards. Ergon's Postgres schema (§**??**) is one such backend; a zip archive written by `ergon export-rollout-card` is another; a HuggingFace dataset shard is a third. The dashboard of Figure **??** is a reference *reader* that loads any of these.

Bundle layout. A card is a logical directory of seven artefacts: `manifest.json` (run-level metadata, format version, content hashes); five append-only JSON-lines streams (`events.jsonl`, `nodes.jsonl`, `edges.jsonl`, `annotations.jsonl`, `mutations.jsonl`); and an optional `blobs/` directory holding content-addressed overflow for payloads above the inline size cap (default 64 KB). The directory may be distributed as-is, packed into a zip or tarball, written to an object-storage prefix, or materialised as a HuggingFace dataset repository with the streams as split files. Nothing in the format assumes Postgres, Python, or any particular runtime.

Row schemas. Each stream is a newline-delimited JSON file; each row has the columns in Table **??**. Column types are JSON primitives (string, integer, object, null); `payload` and `old_value`/`new_value` columns are arbitrary JSON objects whose shape depends on a row-level discriminator (`event_type` for events, `mutation_type` for mutations). The discriminator-specific payload shapes for the Ergon reference implementation are documented in §**??** (events) and Table **??** (mutations).

Table 8: Rollout-card JSONL row schemas. Each stream carries a monotonic `sequence`; its scope (per-run or per-`task_execution_id`) is noted. Discriminator columns select the `payload` shape.

Stream	Row schema
`events.jsonl`	`(event_id, task_execution_id, worker_binding_key, sequence, event_type, turn_id, payload, started_at, completed_at, policy_version)`; `sequence` monotonic per `task_execution_id`.
`nodes.jsonl`	`(node_id, parent_id, instance_key, task_key, status, assigned_worker_key, level, created_at, updated_at)`.
`edges.jsonl`	`(source_node_id, target_node_id, status, created_at, updated_at)`; `status` $\in$ {`pending`, `satisfied`, `invalidated`}.
`annotations.jsonl`	`(target_type, target_id, namespace, sequence, payload, created_at)`; latest `sequence` within `(target, namespace)` is current; prior rows retained.
`mutations.jsonl`	`(sequence, mutation_type, target_type, target_id, actor, old_value, new_value, reason, created_at)`; `sequence` monotonic per run.

Semantics-preserving invariants. Any backend emitting a card must honour four invariants regardless of how rows are stored: (i) `mutations.jsonl` is strictly append-only — reversals appear as new mutations, deletions as tombstones (§**??**); (ii) `events.jsonl` is append-only per `task_execution_id`; (iii) `annotations.jsonl` is namespace-keyed, with the latest `sequence` within a `(target, namespace)` pair as the current value and prior rows retained for replay; (iv) the DAG implied by `edges.jsonl` is acyclic at every point in the run's lifetime (Ergon enforces this at write time via the `edge.added` invariant; §**??**). Cards emitted from Ergon's Postgres backend satisfy these by construction; a third-party backend writing cards directly must enforce them itself.

Extensibility. `manifest.json` carries a format version (`schema: "ergon.tau_e/0.4"`); consumers check the major version and tolerate minor-version additions. Additional columns on existing streams are permitted and must be ignored by consumers that do not recognise them. The public extension point for new metadata is `annotations.jsonl`'s `namespace` field: a consumer claims a namespace, writes domain-specific payloads under it, and the format guarantees that round-tripping a card through a reader that does not understand the namespace preserves those rows intact. One namespace, `ergon.task`, is reserved for task payloads; all others are available to experiments, projections, or external instrumentation.

Reference implementations. The library ships three artefacts. A Pydantic model set plus JSON-schema export for the row types above (`ergon_core/core/persistence/rollout_card/models.py`); a zip-archive exporter (`ergon export-rollout-card <run_id>`) that emits the bundle from any completed Postgres run; and a validator (`ergon validate-rollout-card <path>`) that checks bundle-layout and invariant conformance against any candidate card. The dashboard frontend consumes cards through the same Pydantic models, and therefore renders any conformant bundle — not just Ergon-emitted ones.

D.2 Postgres reference backend

Ergon's reference backend for the rollout-card format of §**??** is a Postgres schema of ten SQLModel tables: one root (`RunRecord`) and nine per-run subtables organised into three layers. Internally, Ergon models a run as a tuple $\tau_E = (N, E, C, A, M)$ — a containment tree of nodes N, a dependency DAG of edges E, a typed reasoning-and-action log C, a namespace-keyed annotation store A, and a mutation log M. The graph layer concretises (N, E, A, M) as four tables; the execution layer concretises C and wraps it in a retry-aware per-attempt wrapper; the communication layer carries inter-agent messages for multi-agent runs. All nine subtables foreign-key to `runs.id`.

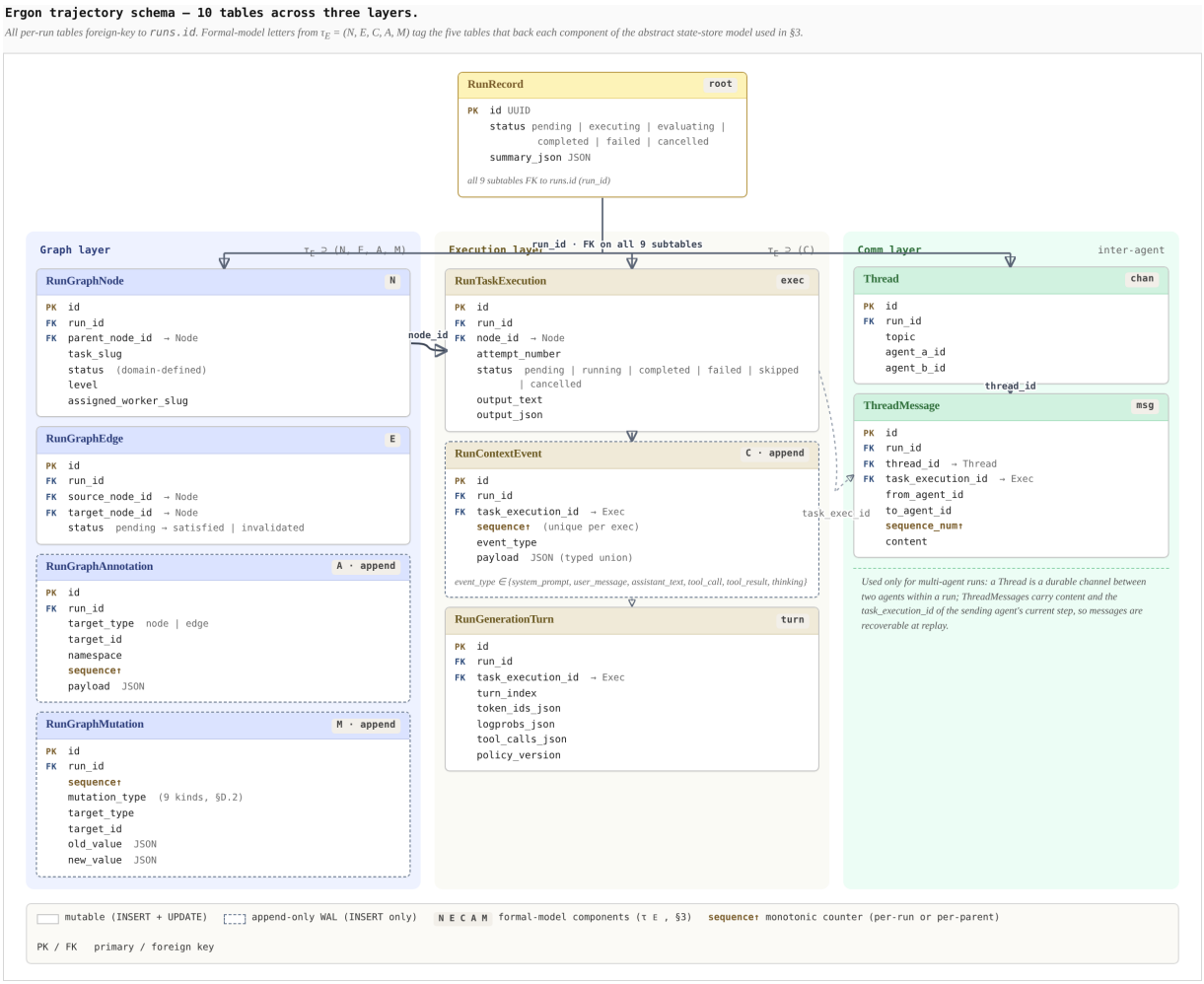

Figure 3: Ergon's ten-table trajectory schema. All per-run tables foreign-key to `runs.id`; the formal-model tags `[N]`, `[E]`, `[C]`, `[A]`, `[M]` identify the five tables that back each component of τ_E. Dashed borders mark append-only write-ahead-log tables (INSERT only); solid borders mark mutable tables (INSERT + UPDATE). The single cross-group foreign key `node_id`: Execution → Node is the join that lets reasoning events and generation turns recover their position in the task DAG.

Figure **??** gives the full topology; schema definitions live in `ergon_core/core/persistence/{graph,context,telemetry}/models.py`.

Graph layer. `RunGraphNode` (`persistence/graph/models.py:L44-L89`) stores the containment tree directly — each node carries its own `parent_node_id` and integer `level`, so the full hierarchy is one indexed SELECT rather than a recursive CTE. `RunGraphEdge` (`:L96-L116`) stores data dependencies with status ∈ {`pending`, `satisfied`, `invalidated`}. `RunGraphAnnotation` (`:L123-L165`) and `RunGraphMutation` (`:L172-L186`) are both append-only: each carries a per-run monotonic `sequence`, and any state at any past point in a run is reconstructed by replaying mutations up to that sequence against an empty graph. Annotations additionally carry a `namespace` so multiple subsystems (experiment config, trainer hints, dashboard state) can version their metadata independently on the same target.

Execution layer. `RunTaskExecution` (`persistence/telemetry/models.py:L96-L157`) wraps one attempt at a node: if a worker retries, each attempt gets its own row with an incremented `attempt_number`, so the context events and generation turns of failed attempts are preserved alongside the successful one. `RunContextEvent` (`persistence/context/models.py:L25-L49`)

is the typed reasoning log — the substrate backing C. It is append-only, with a `sequence` unique per `task_execution_id`, and a discriminated-union `payload` whose shape depends on `event_type`:

system_prompt, user_message: plain `text` (user messages additionally carry `from_worker_key` to attribute inter-worker sends).
assistant_text, thinking: `text`, `turn_id`, and optional `turn_token_ids` / `turn_logprobs` (populated for vLLM, absent for cloud APIs that do not expose token-level information).
tool_call: `tool_call_id`, `tool_name`, `args`, plus the same token/logprob fields.
tool_result: `tool_call_id`, `tool_name`, `result`, `is_error`.

`RunGenerationTurn` (`persistence/telemetry/models.py:L383-L464`) is the per-model-call convenience extraction: one row per call, with the raw response object, an extracted `response_text`, `token_ids_json`, `logprobs_json`, and `tool_calls_json`. It is redundant with `RunContextEvent` — the event log is the substrate, the turn table is a reader-friendly index — but carrying both avoids rehydrating token arrays from event payloads on every training step.

Communication layer. `Thread` (`persistence/telemetry/models.py:L343-L353`) and `ThreadMessage` (`:L360-L376`) are used only by multi-agent runs. A thread is a durable channel between two named agents within a run; each `ThreadMessage` carries the sending agent's current `task_execution_id`, so a replay can recover which reasoning step authored which message. Single-agent runs leave both tables empty.

Engine. The production deployment is PostgreSQL 15; SQLite is used only for test fixtures, which forces JSON payload columns to use the broadly portable `JSON` type rather than `JSONB`. The consequence is that projection code (§**??**) must read JSON payloads client-side rather than push them through Postgres path operators, but this is cheap because projections run at export time, not on the rollout hot path. Migrations are managed by Alembic at `ergon_core/migrations/`.

D.3 Mutation Kinds

Every change to the run graph flows through a single dispatcher, `GraphRepository._log_mutation` (`ergon_core/core/runtime/services/graph_repository.py:L848-L890`), which allocates the next per-run `sequence` (line L848-L856: `SELECT MAX(sequence)+1` within the current run), writes one `RunGraphMutation` row with `old_value`/`new_value` snapshots, and fires registered listeners asynchronously. Mutations are strictly append-only: reversals produce new mutations, not edits to prior ones; deletions are tombstones (see `annotation.deleted` below). The system defines nine mutation kinds (literal declaration at `persistence/graph/models.py:L24-L34`), summarised in Table **??**.

Three invariants are worth calling out because they resolve otherwise-tricky concurrent-write conditions without distributed coordination:

Acyclicity on `edge.added` (`:L892-L915`). Each `edge.added` runs a DFS from `target_node_id` following outgoing edges; if the walk reaches `source_node_id`, the insertion is rejected with `CycleError`. Enforcing this at the repository rather than the experiment layer means every run graph is a DAG by construction, which is what downstream projections ($\pi_{\text{call-tree}}$, $\pi_{\text{per-agent}}$) rely on for their topological walks.

Terminal-write guard on `node.status_changed` (`:L381-L382`). When called with `only_if_not_terminal=True`, the mutation is skipped if the node already holds a terminal status (`COMPLETED`, `FAILED`, or `CANCELLED` per `status_conventions.py:L23`). This single check resolves cascade-cancellation races: concurrent paths that both attempt to write a terminal status converge on first-writer-wins without requiring a distributed lock.

Tombstone semantics on `annotation.deleted` (`:L662-L672`). A "deletion" inserts a new annotation row with an empty payload rather than removing prior rows. This preserves the ability to replay the full annotation timeline — the diagnostic dashboards and the counterfactual-replay tooling both depend on being able to reconstruct every historical payload at any sequence, including the ones that were later cleared.

Table 9: The nine mutation kinds. *Payload* lists the domain-specific fields beyond the common (`sequence, target_type, target_id, actor, old_value, new_value`). *Invariants* are checked in the dispatch path at `runtime/services/graph_repository.py`; when no entry appears, the repository performs no domain validation and defers enforcement to the experiment layer (per the repository contract at `graph_repository.py:L7-L9`).

Kind	Payload	Invariants	File:L-L
`node.added`	`task_slug`, `instance_key`, `description`, `status`, `assigned_worker_slug`	None.	`:L302-L312`
`node.removed`	(same as above)	Cascades `edge.removed` for all incident edges first, then marks node terminal (node rows are not deleted).	`:L314-L358`
`node.status_changed`	`status`	If `only_if_not_terminal=True`, skip if already in {`COMPLETED`, `FAILED`, `CANCELLED`}.	`:L360-L400`
`node.field_changed`	`field`, `value`	`field` ∈ {`description`, `assigned_worker_slug`} (whitelist at `:L62`); else `ValueError`.	`:L402-L434`
`edge.added`	`source_node_id`, `target_node_id`, `status`	Both endpoints must exist (`DanglingEdgeError`); new edge must not create a cycle (DFS at `:L892-L915`).	`:L438-L474`
`edge.removed`	(same as above)	None; edge row marked terminal (not deleted).	`:L476-L502`
`edge.status_changed`	`status`	None at repo layer; lifecycle `pending` → `satisfied` / `invalidated` is experiment-layer policy.	`:L504-L531`
`annotation.set`	`namespace`, `payload`	None; each call inserts a new annotation row (no upsert). Latest `sequence` within namespace is the current value.	`:L535-L573`
`annotation.deleted`	`namespace`, `payload`	Soft delete: inserts a tombstone row with empty payload so the append-only log retains full history.	`:L647-L685`

Edge-status lifecycle. Conventionally `pending` → `satisfied` on dependency resolution and `pending` → `invalidated` on upstream cancellation; backward transitions are not prevented at the repository, but the experiment layer that schedules workers treats `invalidated` as terminal. The string values are constants in `status_conventions.py:L30-L32` and are not enforced by a DB-level check constraint — custom experiment graphs are free to add their own edge statuses, at the cost of being invisible to default dashboards.

D.4 Trainer Adapters

Ergon decouples rollout execution from trainer execution by serving a small HTTP surface that all three supported trainers (TRL, VERL, OpenRLHF) consume. The server is a FastAPI application at `ergon_core/core/api/rollouts.py:L24-L89`; trainer-side adapters are < 100-line shims in `ergon_infra/ergon_infra/adapters/`. The trainers pull; the server never pushes.

Endpoints. Four routes, all under `/rollouts`. `POST /submit` accepts a `SubmitRequest` (`core/rl/rollout_types.py:L21-L35`) carrying `definition_id`, `num_episodes`, `policy_version`, and an optional `model_target_override`; it returns `SubmitResponse` with a `batch_id`, a list of `run_ids`, and the initial `BatchStatus` (status 202). `GET /{batch_id}` returns a `PollResponse` (`:L61-L69`) with `status`, `completed`, `total`, the ordered list of completed `Trajectory` objects, and a list of `EpisodeFailure` records for any runs that crashed. `DELETE /{batch_id}` cancels an in-flight batch. `POST /sync-weights` triggers an optional vLLM reload for full-weight RFT scenarios (`:L67-L88`). Batch and per-episode state are persisted in Postgres (`RolloutBatch`, `RolloutBatchRun`), so trainers can resume polling after either a trainer-side or server-side restart.

Intermediate representation. The `Trajectory` object (`rollout_types.py:L38-L51`) is the one wire-format contract the three adapters share. It is a flat tuple:

(run_id, agent_id, prompt_ids, completion_ids, logprobs, env_mask, reward, num_turns).

It is constructed at poll time by extract_agent_trajectories in core/rl/extraction.py:L49-L117. The extraction walks the rollout's context events C — not its generation turns — so the wire format is grounded in the substrate's append-only log rather than in a denormalised turn view. Concretely: the function groups RunContextEvent rows by worker_binding_key, builds the prompt from the initial system_prompt/user_message events, and then walks the remaining events in sequence order. Model-authored events (assistant_text, tool_call, thinking) contribute their turn_token_ids to completion_ids with env_mask=1 and their turn_logprobs to logprobs; environment events (tool_result) contribute tokenised text to completion_ids with env_mask=0 and zero-padded logprobs. The scalar reward is attached by RewardStrategy.assign (extraction.py:L103) from the run's RunTaskEvaluation rows. Because τ_E is event-sourced, the entire IR is reproducible at export time against the pinned Postgres row versions — the same rollout replayed tomorrow produces byte-identical IR.

Adapters. Each adapter is a shim that calls submit, polls {batch_id}, and maps the returned Trajectory into the native batch type the trainer expects. Table **??** summarises; the longest file is ∼90 lines.

Table 10: Trainer adapters. All three are thin shims over the shared /rollouts endpoints; the framework-specific work is field renaming and wrapping in the native batch type.

Trainer	Adapter file	Output type	Field renames and quirks
TRL (GRPO)	trl_http.py:L25-L91	dict (GRPOTrainer contract)	reward → completion_reward; synchronous httpx.Client; batch of n trajectories per call.
VERL	verl_http.py:L23-L82	AgentLoopOutput	completion_ids → response_ids; env_mask → response_mask; async; single episode per call (streaming integration).
OpenRLHF	openrlhf_http.py :L24-L84	dict (input_ids, response_ids, logprobs, reward)	Drops env_mask (OpenRLHF's return shape does not carry turn masks); module-level configure() for one-time setup; async; single episode per call.

Because the trainers consume the same Trajectory contract, a model trained under one trainer can be evaluated against another trainer's harness without re-exporting: the Ergon substrate is written once, and the format difference is absorbed in the adapter layer. A new trainer is added by writing one adapter file; no changes to the substrate, the server, or the existing adapters are required. The repository ships no SkyRL adapter at submission time, but the pattern supports one at the same cost as the existing three.

E Tech-Stack Integrations

This appendix is a descriptive integrations *map*, not a normative capability matrix. For each of three layers — agent frameworks the substrate hosts as inner runtimes, RL trainers the substrate feeds via HTTP adapters, and canonical trajectory formats the substrate projects to — we list the concrete integrations shipped at submission time, together with the integrations on the near-term roadmap. Rows marked *integrated* are exercised by the experiments in §**??** or by the Ergon test suite; rows marked *planned* are scoped in RFCs in the Ergon repository under docs/rfcs/active/ and do not contribute to any result reported in this paper.

Agents layer. Ergon hosts third-party agent loops as inner runtimes behind an AgentRuntimeAdapter contract: the adapter wraps the framework's loop, serialises each

turn as a `RunContextEvent` (see §**??**), and replays stored events to reconstruct framework-native message state on resumption. One adapter ships today; five are scoped (Table **??**).

Table 11: Agent-framework integrations. Integrated rows are exercised by the flexible-agent worker. Planned rows are scoped in the referenced RFC in the Ergon repository.

Framework	Status	Evidence / RFC
Pydantic AI	integrated	`workers/baselines/react_worker.py:L28-L105` wraps `Agent.iter()`; context replay at `persistence/context/assembly.py:L94-L130`
LangGraph	planned	`agent-framework-adapter-layer` (state-graph replay)
CrewAI	planned	`agent-framework-adapter-layer` (task-delegation shim)
AutoGen	planned	`agent-framework-adapter-layer` (per-agent `worker_binding_key`)
Google ADK	planned	`agent-framework-adapter-layer` (state-machine replay)
Claude Code	planned	`agent-framework-adapter-layer` (closest native fit)

RL trainers. Ergon serves a FastAPI `/rollouts` surface consumed by framework-specific shims; the handshake is documented in Appendix **??**. Three shims ship today — TRL, VERL, OpenRLHF — each ∼80–90 lines. Six are scoped (Table **??**).

Table 12: RL-trainer integrations. Integrated rows are the three adapters compared in Table **??**. Planned rows are scoped in `rl-trainer-adapter-expansion`, which also formalises the HTTP handshake as a `TrainerHttpAdapter` Protocol.

Trainer	Status	Evidence / RFC
TRL (GRPO)	integrated	`adapters/trl_http.py:L1-L92`
VERL	integrated	`adapters/verl_http.py:L1-L83` (`@register("ergon")`)
OpenRLHF	integrated	`adapters/openrlhf_http.py:L1-L85`
SkyRL	planned	`rl-trainer-adapter-expansion`
ProRL Agent	planned	`rl-trainer-adapter-expansion`
AReaL	planned	`rl-trainer-adapter-expansion`
AgentGym-RL	planned	`rl-trainer-adapter-expansion`
RAGEN	planned	`rl-trainer-adapter-expansion`
MARTI	planned	`rl-trainer-adapter-expansion`

Projection / export formats. The five canonical trajectory shapes surveyed in §**??** each correspond to a projection operator over the substrate. Two ship today (step-indexed tuples and per-agent streams, both produced by `extract_agent_trajectories` at `core/rl/extraction.py:L49-L117`); three are scoped (Table **??**). The three planned projections are pure reads over the existing schema (option-tagged, call-tree) or reserve a new annotation namespace (MCTS) — no DB migration is required for any of them.

Of the twenty rows in the three tables, six are integrated and fourteen are planned. The integrated subset is sufficient for the experiments reported in §**??**; the planned subset is what Ergon must ship to fully back the substrate framing of §**??**. Each planned RFC scopes the code additions required and notes the paper-parity dependency explicitly.

F Experimental Setup

This appendix consolidates the experimental-setup details supporting §**??**: per-benchmark agent action spaces (§**??**), benchmark selection and statistics (§**??**), the flexible-agent scaffold used as

Table 13: Projection-operator integrations. The five shapes map to the trajectory-format survey in §**??**. Planned rows are one RFC each.

Projection	**Status**	**Evidence / RFC**
Step-indexed tuples	integrated	`core/rl/extraction.py:L49-L117`
Per-agent streams	integrated	same, keyed by `worker_binding_key`
Option-tagged (semi-MDP)	planned	`projection-operator-option-tagged`
Call-tree (nested)	planned	`projection-operator-call-tree`
MCTS search-tree	planned	`projection-operator-mcts` (`mcts.*` annotation namespace)

the workload-generating system (§**??**), and the cross-harness reconciliation methodology for the SWE-Bench Verified comparison (§**??**).

F.1 Agent action spaces

Table **??** lists the full action space available to the flexible-agent worker on each benchmark. Every benchmark agent has access to the four subtask-decomposition tools (shared across all benchmarks); task-specific tools vary.

Table 14: Per-benchmark agent action space. The four subtask-decomposition tools are shared across all benchmarks; remaining tools are task-specific.

Benchmark	**Tool**	**What it does**
All	`spawn_subtask`	Create a child subtask with a description and required inputs
All	`cancel_subtask`	Cancel a subtask by ID (late cancellation is distinct from completion)
All	`wait_on_subtask`	Block on a subtask ID until it reaches a terminal status
All	`report_result`	Write this task's result and mark the task complete
MiniF2F	`lean_repl`	**[TODO: short description]**
MiniF2F	`lean_check`	**[TODO: short description]**
Research Rubrics	`web_search`	**[TODO: short description (Tavily)]**
Research Rubrics	`read_document`	**[TODO: short description (httpx+trafilatura)]**
SWE-Bench Verified	`bash`	**[TODO: short description]**
SWE-Bench Verified	`edit_file`	**[TODO: short description]**

The subtask-decomposition tools are exposed uniformly: the worker's system prompt describes them mechanically and gives no guidance on when to use them, so observed delegation structure comes from the agent rather than from prompting (see §**??**).

F.2 Benchmark details

Table A2: MiniF2F problem selection. *Difficulty levels, problem categories, proof-length statistics, subset used in §**??**.*

Table A3: Research Rubrics question distribution. *Topic distribution, rubric criteria counts, evaluator configuration.*

Table A4: SWE-Bench Verified subset. *Per-repository counts in the 100–150 instance subset used in §**??**, sampling procedure, comparison to full Verified statistics, E2B template details.*

F.3 Flexible agent and projection details

F.3.1 Benchmark delegation toolkit

The flexible-agent worker of §**??** is given a particular set of delegation actions that mutate the substrate. This action space is a design choice of our benchmark, not a feature of Ergon itself — another research team could expose a different set of delegation actions against the same substrate and run a different benchmark. The toolkit we expose consists of six verbs, each corresponding to a named mutation pattern on (N, E, A):

- `add_subtask`: creates a single child node and dispatches it for asynchronous execution.
- `plan_subtasks`: atomically creates a sub-DAG of children validated by Kahn's algorithm, with root children dispatched to a concurrency-15 executor.
- `cancel_task`: marks a node terminal, cascades to descendants, and invalidates outgoing dependency edges (cancelled sub-trees remain in (N, E) for analysis).
- `refine_task`: edits a non-running node's description with field-history preserved.
- `restart_task`: resets a terminal node to PENDING and cascades invalidation to downstream dependencies.
- `list_subtasks` and `get_subtask`: read-only observations of the current containment tree.

Each verb maps to a named mutation type in M, so downstream evaluators distinguish delegation, cancellation, and refinement from ordinary tool use without custom parsing — independently of what specific verbs our benchmark chose to expose.

F.3.2 Agent details and projections

[TODO: For the flexible-agent worker of §??: system prompt, full tool inventory (benchmark action space above plus task-specific tools), one-shot example if any, and the turn-budget and stopping rules. For each of the five projection operators π_{step}, $\pi_{\text{per-agent}}$, $\pi_{\text{call-tree}}$, π_{macro}, $\pi_{\text{json-log}}$: one worked example on one trajectory from each task family, the drops manifest it produces, and discussion of which behavioural metrics each projection structurally cannot preserve.]

F.4 Cross-harness reconciliation methodology

[TODO: Session 11: Cross-harness reconciliation methodology appendix. Source: Weekend 1 Sec. 4.5 experiment. Structure: (a) convention specification — union convention justified against alternatives (SWE-agent's convention, Agentless's convention, minimal common denominator); (b) download and re-grading code walkthrough; (c) what the re-grading preserves vs what it cannot preserve (explicit drops manifest for the reconciliation convention itself); (d) drops

manifests for the SWE-agent → rollout-card and Agentless → rollout-card ingestions per Sec. 4.2 "Cross-harness reconciliation"; (e) `no_generation` convention analysis — 50 instances SWE-agent, 4 Agentless; why Convention A (exclude) and Convention B (include as zero) bracket the reasonable range of harness-level choices.]

G Behavioural Quantities

This appendix catalogues the six behavioural quantities referenced in §**??**. Each is a canonical analysis of one of the five research communities surveyed in §**??**. The set is illustrative: a rollout card supports arbitrarily many analyses, and the three quantities exercised in §**??** (abandonment ratio by depth, per-agent sibling embedding distance, and the `no_generation` split in the SWE-bench reconciliation) are drawn from this set.

Table 15: Six behavioural quantities exercised in §**??**, each the canonical analysis of one of the five research communities from §**??**. The set is illustrative: a rollout card supports arbitrarily many such analyses.

Quantity	**Community**	**Computed from**
Step-indexed return under episode mask	Long-horizon LLM-agentic RL (**??**)	`events.jsonl` token spans
Option termination frequency	HRL / macro-action (**?**)	`mutations.jsonl` status transitions
Call-tree depth distribution	Recursive LMs (**?**)	`nodes.jsonl` `parent_id` chains
Per-agent sibling embedding distance	Fixed-role MAS (**?**)	`nodes.jsonl` + `events.jsonl`
Node visit entropy across frontier	MCTS-based training (**??**)	`annotations.jsonl` (MCTS namespace)
Dispatch-and-wait rate	Async MAS	`events.jsonl` + `mutations.jsonl`

H Benchmark Card: Evaluative Role, Assumptions, and Limitations

Evaluative role. The Ergon benchmark suite, paired with the trajectory representation and projection operators, supports evaluative claims about dynamic-delegation agents that were previously not comparable across research communities: (i) whether an agent uses concurrent dispatch when it is available; (ii) whether an agent cancels sub-tasks that are not making progress; (iii) whether an agent's delegation structure contains dependency diamonds that exceed tree-shape formalisms; (iv) whether the agent's reasoning expresses intent that its recorded structural actions fail to realise; (v) whether a community's preferred trajectory format preserves each of these behaviours under its projection operator.

Assumptions.

- The tool implementations (web search, document retrieval, Lean REPL, SWE-Bench harness) are stable at their released versions and latency distributions; substantial drift in tool-provider behaviour may invalidate fault-injection calibration.
- The rubric-based evaluator (GPT-4o-mini, for Research Rubrics) is treated as a noisy but calibrated judge; we do not claim evaluator infallibility and provide inter-rater agreement statistics in Appendix **??**.
- The flexible-agent worker is given access to the subtask-decomposition tools but not prompted to use them; observed delegation structure reflects the backbone model's untutored decomposition under a minimal system prompt, not a researcher-imposed strategy.

Limitations.

- Three task families; generalisation claims to other long-horizon settings should be made cautiously. The behavioural coverage of §**??** is complementary but not exhaustive.

- LLM-judge failure modes (reward hacking, distribution shift) are documented but not fully characterised.
- Tool-provider API drift may require re-calibration; reproducibility bundle pins tool-provider snapshots. SWE-Bench Verified's per-repository environment specs are pinned to the `swebench` package version used.

Intended use. Evaluation of dynamic-delegation policies via the substrate and projection-preservation framework. The benchmark suite is *not* intended as a stand-alone prover, a factual QA benchmark, or a general agentic capability test; claims in those directions should use purpose-built benchmarks and metrics.

Failure modes. Rubric reward hacking on Research Rubrics; MiniF2F proof leakage from the backbone's training data; SWE-Bench test-pattern overfit on popular repositories; tool-provider API drift; LLM-judge sycophancy.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and §**??** claim (i) that current agent research publishes reported numbers without rollouts, producing cross-community fragmentation and cross-harness variance, and (ii) that rollout cards plus drops manifests address both problems; these are substantiated by the 50-repo survey (Appendix **??**), the 37-pair variance catalogue (Appendix **??**), the format specification in §**??** and Appendix **??**, and the two experiments of §**??**. Scope limits (proof-of-concept, two RQ1 pairings, one RQ2 reconciliation on a single benchmark, two public submissions) are stated explicitly in §**??** and revisited in the Limitations paragraph of §**??**.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: §**??** contains an explicit Limitations paragraph that enumerates: representative coverage of five communities from a long tail, 50 repositories from a much larger population, 37 variance pairs from the documented-comparison subset; citation-based (not reproduced) training-side transform-variance evidence; single-analyst cross-community reanalysis (one target community per task family); and single-benchmark reconciliation (SWE-bench Verified, two published submissions). The paper's claims are scoped to these bounds.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.

- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not state theorems or prove formal results. Projection operators in Appendix **??** are defined constructively as pseudocode specifications with accompanying drops manifests; preservation claims are enumerative (Table **??**) and follow directly from the projection definitions, not from proof obligations.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: §**??** specifies the three task families, agent backbone, task-specific tools, and grading procedures; Appendix **??** consolidates the per-benchmark action space (§**??**), benchmark selection (§**??**), flexible-agent details (§**??**), and cross-harness reconciliation methodology (§**??**); Appendix **??** specifies the rollout-card format bit-for-bit. The reference implementation is open-source; all reported numbers are either deterministic functions of published rollouts (RQ2) or derivable from the released rollout cards (RQ1).

   Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Ergon (the recording substrate and reference implementation) is released as open source with README, architecture documentation, and benchmark harness; an anonymised artefact bundle accompanies this submission. The SWE-agent and Agentless SWE-bench Verified submissions used in RQ2 are already public in the `swe-bench-submissions` S3 bucket. Rollout cards for the MiniF2F and Research Rubrics experiments will be released alongside the paper, as will the cross-harness reconciliation pipeline and convention specification (§**??**).

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.

- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper reports no training; all rollouts are inference-only against an API-served model. §**??** specifies the model backbone, per-task-family turn cap, action space, and grading procedure. Appendix **??** specifies the benchmark subsets used (§**??**), the full agent tool inventory and system-prompt structure (§**??**), and the convention choices in the SWE-bench reconciliation (§**??**).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The RQ2 cross-harness reconciliation is a deterministic re-grading of already-released trajectories, so no stochastic variation enters the pipeline; we report exact denominators and decomposition deltas rather than error bars. For the RQ1 rollouts (MiniF2F and Research Rubrics), we report Wilson-score 95% confidence intervals for the rate statistics (abandonment ratios, role-differentiation fractions) and specify the rollout count driving each interval; the cross-community analyses are qualitative proof-of-concept claims at the single-seed level, which is stated explicitly.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).

- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: No model training is performed. Agent rollouts for RQ1 are inference-only calls to an API-hosted LLM backbone; rollout counts and per-rollout token budgets are reported in Appendix **??**. The RQ2 reconciliation pipeline (ingestion of SWE-agent and Agentless submissions, re-grading under two conventions) runs on a single developer workstation in minutes; compute requirements are stated in §**??**. No GPU is required for any reported experiment.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The paper proposes a publication format and reference implementation for agent research. It involves no human subjects and no data scraping beyond publicly-released benchmark submissions whose licenses permit re-analysis. The reference implementation is released as open source.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Positive impacts — improved auditability of agent-research claims, cross-harness reconciliation, and legibility of methodology differences — are discussed in §**??** ("Publication cost and adoption" and "Detection, not resolution"). Negative impacts are minimal: a publication format itself cannot be weaponised, and the rollouts released alongside benchmarks inherit the data-handling norms of their source benchmarks (MiniF2F, SWE-bench Verified, Research Rubrics). No new scraped or high-risk data is introduced.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases a format specification (rollout cards) and a reference recording substrate (Ergon). Neither is a pre-trained model, generative system, or scraped dataset. The rollouts released alongside are bounded by the source benchmarks' existing release policies.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: MiniF2F (**?**), SWE-bench Verified, SWE-agent and Agentless submissions, the Research Rubrics benchmark, and the LLM API backbones used for rollout generation and rubric grading are all cited in §**??** and §**??**. Asset licenses and version identifiers are catalogued in Appendix **??**, including the specific SWE-bench submission commits and the API model versions used.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: Three new assets accompany the paper: (i) the rollout-card format specification (Appendix **??**); (ii) Ergon, the reference implementation, with a README, architecture documentation, and RFC stream for pending integrations (Appendix **??**); (iii) ingestion adapters for SWE-agent and Agentless submissions with accompanying drops manifests (§**??**). All three are documented in-repository and anonymised for submission.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no human subjects. The Research Rubrics LLM-based evaluator is an API grading transform, not a human participant; the 50-repo survey and the 37-pair variance catalogue are authored by the paper's authors against public artefacts.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects research is performed. No IRB review is applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are the central object of study. The flexible-agent worker of §**??** is an LLM-backed scaffold running on an API-hosted backbone; the Research Rubrics evaluator is a GPT-4o-mini grading transform; the SWE-bench reconciliation analyses trajectories produced by third-party LLM agents (SWE-agent, Agentless) on a common API-hosted backbone. All model identities, versions, and prompting interfaces are named in §**??** and Appendix **??**.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.